# HIRELENS AI - EXPLAINABLE AI RESUME SCREENING SYSTEM

*A Project Phase – II Report Submitted*

*In partial fulfillment of the requirement for the award of the degree of*

**Bachelor of Technology**
***in***
**Computer Science and Engineering**
**(Artificial Intelligence and Machine Learning)**

**by**


| | |
|---|---|
| **BHEEMANAPALLY ABHAYA SRI** | **22N31A6627** |
| **BOCHKAR NIKHITH** | **22N31A6629** |
| **JAKKIREDDY SRI CHARAN REDDY** | **22N31A6666** |


***Under the esteemed Guidance of***

**Mr. S. Venkateswara Raju**
**Assistant Professor**


**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**
**(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)**
**MALLA REDDY COLLEGE OF ENGINEERING AND TECHNOLOGY**
**(Autonomous Institution - UGC, Govt. of India)**
**(Affiliated to JNTU, Hyderabad, Approved by AICTE, Accredited by NBA & NAAC – 'A' Grade, ISO 9001:2015 Certified)**
Maisammaguda (v), Near Dullapally, Via: Kompally, Hyderabad – 500 100, Telangana State, India. website: www.mrcet.ac.in


**2025-2026**

# DECLARATION

We hereby declare that the project entitled **"HireLens AI – Explainable AI Resume Screening System "** submitted to **Malla Reddy College of Engineering and Technology,** affiliated t**o** Jawaharlal Nehru Technological University Hyderabad (JNTUH) for the award of the degree of **Bachelor of Technology** in **Computer Science and Engineering- Artificial Intelligence and Machine Learning** is a result of original research work done by us.

It is further declared that the project report or any part thereof has not been previously submitted to any University or Institute for the award of degree or diploma.

**Bheemanapally Abhaya Sri (22N31A6627)**
**Bochkar Nikhith (22N31A6629)**
**Jakkireddy Sri Charan Reddy (22N31A6666)**

# CERTIFICATE

This is to certify that this is the bonafide record of the project titled **"HireLens AI – Explainable AI Resume Screening System"** submitted by **Bheemanapally Abhaya Sri** (22N31A6627), **Bochkar Nikhith** (22N31A6629), **Jakkireddy Sri Charan Reddy** (22N31A6666) of B.Tech in the partial fulfillment of the requirements for the degree of **Bachelor of Technology** in **Computer Science and Engineering- Artificial Intelligence and Machine Learning**, Dept. of CI during the year 2025-2026. The results embodied in this project report have not been submitted to any other university or institute for the award of any degree or diploma.

**Mr. S. Venkateswara Raju**
Assistant Professor

**INTERNAL GUIDE**

**Dr. A. Nagaraju**
Professor

**HEAD OF THE DEPARTMENT**

**EXTERNAL EXAMINER**

**Date of Viva-Voce Examination held on: _____**

# ACKNOWLEDGEMENT

The Project Phase I work carried out by our team in the Department of CSE (Artificial Intelligence and Machine Learning), Malla Reddy College of Engineering and Technology, Hyderabad. ***This work is original and has not been submitted in part or full for any degree or diploma of any other university.***

We wish to acknowledge our sincere thanks to our project guide ***Mr. S. Venkateswara Raju***, Assistant Professor for formulation of the problem, analysis, guidance and his continuous supervision during work.

We acknowledge our sincere thanks to ***Dr. Vaka Murali Mohan***, Principal, ***Dr. A. Nagaraju***, Head of the Department and ***Dr. T. Siva Ratna Sai***, Coordinator, faculty members of CSE(AI&ML) Department for their kind cooperation in making this Project Phase I work a success.

We extend our gratitude to ***Sri. Ch. Malla Reddy***, Founder Chairman MRGI and ***Sri. Ch. Mahender Reddy***, Secretary MRGI, ***Dr. Ch. Bhadra Reddy***, President MRGI, ***Sri. Ch. Shalini Reddy***, Director MRGI, ***Sri. P. Praveen Reddy***, Director MRGI, for their kind cooperation in providing the infrastructure for completion of our Project Phase I.

We acknowledge our special thanks to the entire teaching faculty and non-teaching staff members of the CSE (Artificial Intelligence and Machine Learning) Department for their support in making this project work a success.

**By**

**Bheemanapally Abhaya Sri (22N31A6627)** _____

**Bochkar Nikhith (22N31A6629)** _____

**Jakkireddy Sri Charan Reddy (22N31A6666)** _____

# ABSTRACT

HireLens AI is an Explainable Artificial Intelligence–based resume screening system designed to make recruitment faster, transparent, and more data-driven. Most existing Applicant Tracking Systems (ATS) work as black boxes, providing hiring decisions without any reasoning, which creates confusion for both candidates and recruiters. HireLens AI solves this by using Natural Language Processing (NLP) and machine learning to extract key details like skills, education, work experience, projects, and certifications from resumes, then comparing them against job descriptions to generate a compatibility score along with clear, interpretable explanations of matched skills, missing qualifications, and relevant experience. It also includes a side-by-side candidate comparison feature to help recruiters make better-informed decisions. The expected outcome is an AI-powered recruitment tool that brings efficiency and fairness to the hiring process, helping recruiters choose the right candidates while giving applicants clear insight into their evaluation.

# TABLE OF CONTENTS

## CONTENTS Page No.


**1. INTRODUCTION 1**

1.1. Problem Statements 1

1.2. Objectives 2

**2. LITERATURE SURVEY 4**

2.1. Existing System 7

2.2. Proposed System 8

**3. SYSTEM REQUIREMENTS 9**

3.1. Software and Hardware Requirement 9

3.2. Functional and Non-Functional Requirements 11

3.3. Other Requirements 12

**4. SYSTEM DESIGN 14**

4.1. Architecture Diagram 14

4.2. UML Diagrams 15

**5. IMPLEMENTATION 19**

5.1. Algorithms 19

5.2. Source Code 21

5.3. Architectural Components 66

5.4. Feature Extraction 68

5.5. Packages/Libraries Used 69

5.6. Output Screens 71

**6. SYSTEM TESTING 76**

6.1. Test Cases 76

6.2. Results and Discussions 78

6.3. Performance Evaluation 79

**7. CONCLUSION & FUTURE ENHANCEMENTS 81**

**8. REFERENCES 83**

# LIST OF FIGURES


| Fig. No | Figure Title | Page no. |
|---|---|---|
| 4.1 | Architecture Diagram | 14 |
| 4.2 | Use Case Diagram | 15 |
| 4.3 | Class Diagram | 16 |
| 4.4 | Sequence Diagram | 17 |
| 4.5 | Activity Diagram | 18 |
| 5.1 | Output 1 – HireLens AI Landing page | 71 |
| 5.2 | Output 2 – Upload Resume Page | 71 |
| 5.3 | Output 3 – Job Title and Job Description page-1 | 72 |
| 5.4 | Output 4 - Job Title and Job Description page-2 | 72 |
| 5.5 | Output 5 – Screening Results page-1 | 73 |
| 5.6 | Output 6 – Screening Results page-2 | 73 |
| 5.7 | Output 7 – Results page | 74 |
| 5.8 | Output 8 – Resume comparison page | 74 |
| 5.9 | Output 9 – Rankings page | 75 |


# LIST OF TABLES


| S. No | Table Name | Page No |
|---|---|---|
| 6.1 | Sample test cases | 76 |


# LIST OF ABBREVIATIONS

| Serial No. | Abbreviations |
|---|---|
| 1 | **AI** – Artificial Intelligence |
| 2 | **XAI** – Explainable Artificial Intelligence |
| 3 | **UML** – Unified Modeling Language |
| 4 | **IE** – Information Extraction |
| 5 | **NER** – Named Entity Recognition |
| 6 | **ATS** – Applicant Tracking System |
| 7 | **API** – Application Programming Interface |
| 8 | **ORM** – Object-Relational Mapping |
| 9 | **GDPR** – General Data Protection Regulation |

# CHAPTER 1
# INTRODUCTION

HireLens AI is a web-based resume screening and candidate evaluation system that integrates Explainable Artificial Intelligence (XAI) principles into the hiring process. The platform enables recruiters and hiring managers to upload candidate resumes and job descriptions, match them algorithmically across multiple weighted dimensions, and receive transparent, human-readable explanations for every screening decision. Built on a modern full-stack architecture comprising a React 19 frontend, a FastAPI Python backend, and a modular AI pipeline, HireLens AI bridges the gap between sophisticated automated screening and the interpretability requirements of real-world hiring workflows.

The system was developed in recognition of the growing need for AI-assisted recruitment tools that do not function as impenetrable "black boxes." Unlike conventional applicant tracking systems that silently rank resumes with opaque heuristics, HireLens AI makes its decision logic visible at every level — from the raw feature scores across skills, experience, education, and project relevance, to a natural language summary that explains precisely why a candidate has been predicted as "Selected" or "Rejected." This commitment to transparency ensures that every hiring decision is data-driven, auditable, and fair.

## 1.1 Problem Statements

The modern recruitment process suffers from a critical scalability and fairness problem. Organizations frequently receive hundreds to thousands of applications for a single job posting, yet the manual review process remains labour-intensive, inconsistent, and susceptible to unconscious bias. Studies in human resource management consistently demonstrate that human recruiters spend an average of six to eight seconds reviewing a single resume at the initial filtering stage, leading to highly superficial first-pass decisions. The consequences are significant: qualified candidates are routinely overlooked, while technically unqualified but well-formatted resumes may pass through screening, wasting recruiter time at later interview stages.

Existing automated solutions, such as commercial Applicant Tracking Systems (ATS), address volume but introduce a new problem: opacity. These systems apply keyword-matching or proprietary machine learning models whose decision criteria are unknown to both the recruiter

and the candidate. A candidate may be rejected without any understanding of which specific qualifications were found to be insufficient. Recruiters themselves cannot audit or calibrate these black-box decisions, making it difficult to defend hiring choices in environments where compliance, diversity, and explainability are regulatory or ethical requirements. There is, therefore, a pressing need for an automated resume screening system that combines comprehensive multi-dimensional analysis with full, human-readable decision transparency — a need that HireLens AI directly addresses.

## 1.2 Objectives

The primary objective of this project is to design, develop, and deploy a fully functional, explainable AI resume screening system capable of accurately evaluating candidate suitability for a given job role and presenting all evaluation reasoning to the end user in a clear and actionable format. This objective encompasses both the technical soundness of the AI pipeline and the usability of the interface through which results are communicated.

- **Automated Multi-Format Resume Parsing:**
  To develop a parsing engine capable of extracting structured information — including name, email, phone number, skills, education, years of experience, projects, and certifications — from resume files in PDF, DOCX, and TXT formats, without requiring manual data entry by the recruiter.

- **Weighted Multi-Dimensional Feature Scoring:**
  To implement a job description parser that identifies required skills, preferred skills, minimum experience thresholds, and educational requirements from free-form textual job postings, converting unstructured recruiter intent into a computable profile.

- **Explainable Decision Generation:**
  To implement an explainability engine that, for every screening outcome, produces feature contribution breakdowns, natural language summaries, matched and missing skill lists, and structured lists of candidate strengths and weaknesses, providing full transparency into each decision.

- **Candidate Ranking and Batch Processing:**
  To provide batch-matching functionality that processes an entire pool of uploaded resumes against a single job description and ranks all candidates by their computed match score in descending order, assisting recruiters in efficiently prioritising top

candidates.

- **Side-by-Side Candidate Comparison:**
  To enable a structured comparison of any two candidates for the same role, presenting their scores, pros, cons, matched and missing skills, and an AI-generated natural language recommendation as to which candidate is the stronger fit.

- **Persistent Data Management:**
  To architect a relational database layer using SQLite and SQLAlchemy that reliably stores all uploaded resumes, job descriptions, and screening results, enabling recruiters to access historical data and revisit past evaluations without re-processing.

- **Responsive Web Interface:**
  To deliver a modern, single-page application frontend built with React and TailwindCSS that provides an intuitive multi-step workflow for resume upload, job description submission, result review, and candidate comparison.

# CHAPTER 2

# LITERATURE SURVEY

- The development of intelligent resume screening systems sits at the intersection of Natural Language Processing (NLP), Information Extraction (IE), machine learning, and Human-Computer Interaction (HCI). The following survey covers key themes, technologies, and systems that informed the design of HireLens AI.

- Resume Parsing Using Information Extraction Methods: Early work in automated resume processing, such as that by Yu et al. (2005), demonstrated that Named Entity Recognition (NER) and rule-based segmentation could reliably identify structured fields like name, contact information, and educational background from plain-text resumes. These techniques established the foundational approach of treating a resume as a semi-structured document amenable to pattern matching.

- Keyword-Based Applicant Tracking Systems (ATS): Commercial ATS platforms such as Workday, Greenhouse, and Taleo pioneered automated first-pass filtering using keyword matching against job description terms. While effective for volume reduction, research by Bogen and Rieke (2018) documented significant false-negative rates, particularly for candidates who use synonymous terminology or who possess transferable skills not anticipated by the keyword list.

- Semantic Similarity for Resume-Job Matching: The use of word embeddings (Word2Vec, GloVe) and sentence embeddings (Universal Sentence Encoder, Sentence-BERT) for computing semantic similarity between resume content and job descriptions represents a significant advance over keyword matching. Sinha et al. (2021) demonstrated that cosine similarity between Sentence-BERT embeddings consistently outperformed TF-IDF keyword overlap on resume-job matching benchmarks. The sentence-transformers library, listed in the project's requirements, provides this capability.

- spaCy for NLP Pipelines: spaCy is an industrial-strength NLP library that provides tokenisation, part-of-speech tagging, dependency parsing, and NER through a unified pipeline API. Its efficiency and accuracy have made it the standard choice for information extraction from professional documents. The project includes spacy==3.7.6 as a core dependency, intended for text preprocessing and entity extraction tasks.

- Multi-Criteria Decision Making in Hiring: Research by Obinna et al. (2020) in the context of multi-criteria candidate evaluation found that weighting recruitment attributes — skills, experience, education, and references — differently based on role type produced significantly more accurate shortlisting outcomes than uniform weight approaches. HireLens AI operationalises this finding through configurable MATCHING_WEIGHTS in its backend configuration.

- Explainable AI (XAI) Principles: The need for explainability in AI-driven decisions is formalised in the European Union's GDPR Article 22, which grants individuals the right to an explanation for automated decisions affecting them. Ribeiro et al.'s seminal paper "Why Should I Trust You?" (2016) introduced LIME (Local Interpretable Model-agnostic Explanations), which is included in the project dependencies (lime==0.2.0.1), as a method for generating post-hoc explanations for individual predictions.

- SHAP Values for Feature Attribution: Lundberg and Lee (2017) introduced SHAP (SHapley Additive exPlanations) as a unified framework for interpreting machine learning model outputs using cooperative game theory. The library shap==0.46.0 is included in the project, offering a mathematically grounded method for attributing each feature's contribution to the final screening decision, thereby satisfying both technical and regulatory explainability requirements.

- KeyBERT for Keyword Extraction: KeyBERT, built on top of BERT-based embeddings, extracts the most relevant keywords from a document by measuring the cosine similarity between word/phrase embeddings and a document's embedding. The library keybert==0.8.5 is included in the project's requirements and would serve to identify the most important technical terms in both resumes and job descriptions, improving skill extraction fidelity.

- Feature Extraction via Ontology-Based Skill Matching: Sager et al. (2018) demonstrated that maintaining a curated ontology of technical skills and computing set intersection between a candidate's skill set and the job's required skill set provides a highly reliable and interpretable measure of technical fit. HireLens AI implements this approach through its KNOWN_SKILLS dictionary containing over 150 normalised technical and domain-specific skills.

- Education-Level Hierarchical Scoring: Existing literature on automated screening suggests encoding education levels as ordinal values in a hierarchy (Diploma < Bachelor's

< Master's < PhD) and computing the ratio of the candidate's level to the required level as a feature score. This approach, formalised by Faliagka et al. (2012), is precisely reflected in the _compute_education_score() function within the feature extractor module.

- FastAPI as a High-Performance Backend Framework: FastAPI, based on Starlette and Pydantic, enables the rapid development of type-safe REST APIs with automatic OpenAPI documentation generation. Its asynchronous capabilities make it particularly well-suited for I/O-bound operations such as file uploads and database queries. The project uses fastapi==0.115.0 as its backend framework.

- SQLAlchemy ORM for Data Persistence: SQLAlchemy provides a declarative ORM layer for Python that abstracts database interactions, enabling clean model definitions and query construction. Its use with SQLite provides a zero-configuration, file-based relational database (hirelens.db) that is appropriate for the scale of an academic prototype while being straightforwardly replaceable with PostgreSQL or MySQL in production.

- React as a Single-Page Application Framework: Modern frontend development research consistently advocates for component-based architectures that enable state-driven UI rendering. React's virtual DOM diffing algorithm and unidirectional data flow model make it ideal for building dynamic, data-intensive applications like HireLens AI, where UI state changes rapidly in response to backend AI results.

- Vite as a Frontend Build Tool: Vite leverages native ES modules and Rollup under the hood to deliver near-instantaneous hot module replacement (HMR) during development and optimised production builds. Its significantly faster development server compared to Webpack-based tools makes it the modern standard for React application scaffolding, as evidenced by its adoption in this project (vite==8.0.0).

- Weighted Linear Combination for Score Aggregation: Triantaphyllou (2000) surveyed weighted linear combination (WLC) as one of the most interpretable and robust multi-criteria aggregation methods. By assigning predetermined weights to each evaluation dimension and computing a weighted sum, WLC produces a single composite score that is both easy to explain and mathematically principled. This is exactly the scoring model implemented in HireLens AI's match_resume_to_job() function, where the final match score is the weighted sum of skill (40%), experience (25%), project (20%), and education (15%) scores.

- Regex-Based Information Extraction: For structured fields such as email addresses, phone

numbers, and years of experience, regular expression (regex) matching provides a deterministic, computationally cheap, and highly accurate extraction method. HireLens AI's resume parser extensively uses compiled regex patterns for these fields, providing reliable extraction without requiring a large language model.

## 2.1 Existing System

The current landscape of automated recruitment tools is dominated by commercial Applicant Tracking Systems (ATS) such as Workday Recruiting, SAP SuccessFactors, Greenhouse, and Lever. These platforms integrate resume parsing (typically powered by third-party vendors such as Sovren or Textkernel), keyword-based filtering, and candidate pipeline management into enterprise HR suites. While operationally efficient at scale, these systems share a fundamental limitation: their decision-making process is opaque. When a candidate is filtered out, neither the recruiter nor the candidate receives a detailed breakdown of which specific criteria caused the rejection. The scoring models are proprietary, the weights are undisclosed, and no feature-level contribution analysis is provided. This opacity creates compliance risks under regulations like the EU GDPR and the US Equal Employment Opportunity guidelines, and it undermines recruiter trust in the system's outputs.

Furthermore, existing systems are typically gated behind expensive enterprise licensing models, making them inaccessible to small and medium-sized businesses, educational institutions, or individual hiring managers. Open-source alternatives such as OpenCATS or Recruiterbox offer basic pipeline management but lack any AI-driven scoring or explainability features. Academic implementations of AI screening tools, while technically sophisticated, are rarely deployable as end-to-end user-facing systems with integrated web interfaces, persistent storage, and real-time result presentation. The gap between research-grade NLP models and practical, usable screening tools remains significant, representing the primary motivation for HireLens AI.

From a technical standpoint, most existing systems rely on either pure keyword extraction or, at best, TF-IDF-based relevance scoring. Neither approach natively supports multi-dimensional weighted evaluation (where skills, experience, education, and project relevance are independently scored and then combined), nor do they generate explanations that identify specifically which matched and missing skills contributed to the final decision — capabilities that HireLens AI provides as core features of its AI pipeline.

## 2.2 Proposed System

HireLens AI addresses the shortcomings of existing systems through a purpose-built, open, explainable AI pipeline tightly integrated with a modern web application. The proposed system replaces black-box scoring with a transparent, weighted multi-dimensional evaluation framework in which every component of the final match score is independently computed, individually attributable, and communicated to the user in both numerical and natural language form.

The system introduces a six-stage AI pipeline: (1) multi-format resume parsing, (2) job description analysis, (3) feature extraction and normalisation, (4) weighted score aggregation, (5) selection/rejection prediction based on a configurable threshold (60%), and (6) explainability generation. Each stage is implemented as an independent Python module within the ai/ directory, ensuring modularity and testability. The backend is exposed as a RESTful API via FastAPI, enabling clean separation of concerns between the AI logic and the web presentation layer.

Critically, the proposed system makes its reasoning accessible at every level. The ExplanationPanel React component on the frontend renders the feature contributions, natural language summary, matched skills, missing skills, strengths, and weaknesses that the explainability module generates for every screening decision. This means a recruiter can instantly understand not just a candidate's score, but the precise reasoning behind it — without any domain knowledge of AI or machine learning. Additionally, the candidate comparison module (comparator.py) provides a structured side-by-side evaluation of two candidates, automatically generating a natural language recommendation that identifies the stronger candidate and explains the basis for the recommendation. These capabilities collectively make HireLens AI a uniquely transparent, auditable, and practically usable AI recruitment tool.

# CHAPTER 3
# SYSTEM REQUIREMENTS

The system requirements for HireLens AI encompass the hardware infrastructure necessary to host the application, the software ecosystem on which it is built, and the functional and non-functional properties the system must exhibit to meet its design objectives. These requirements were identified through a combination of analysis of the codebase, its dependencies, and the operational characteristics of the AI processing pipeline.

## 3.1 Software and Hardware Requirements

To develop and run the application efficiently, the following software and hardware specifications have been identified.

**Software Requirements:**

**Backend:**

- Python 3.10 or higher (required for type annotation compatibility and modern library support)
- FastAPI 0.115.0 — Web framework for the REST API
- Uvicorn 0.30.0 (with standard extras) — ASGI server for running the FastAPI application
- SQLAlchemy 2.0.35 — Object-Relational Mapper for database interaction
- SQLite (bundled with Python) — Relational database for persisting resumes, jobs, and results
- pdfplumber 0.11.4 — PDF text extraction library
- python-docx 1.1.2 — DOCX document text extraction library
- python-multipart 0.0.9 — Multipart form data handling for file uploads
- spaCy 3.7.6 — Industrial-strength NLP library for text preprocessing
- sentence-transformers 3.1.0 — Sentence embedding models for semantic similarity
- scikit-learn 1.5.2 — Machine learning utilities including cosine similarity
- SHAP 0.46.0 — SHapley Additive exPlanations for model interpretability
- LIME 0.2.0.1 — Local Interpretable Model-agnostic Explanations
- KeyBERT 0.8.5 — BERT-based keyword extraction

- NumPy 1.26.4 — Numerical computing library
- Pydantic (bundled with FastAPI) — Data validation and schema definition

**Frontend:**

- Node.js 18+ and npm — JavaScript runtime and package manager
- React 19.2.4 — UI component library
- React DOM 19.2.4 — React rendering for web browsers
- React Router DOM 7.13.1 — Client-side routing
- Vite 8.0.0 — Frontend build tool and development server
- TailwindCSS 4.2.1 — Utility-first CSS framework
- Axios 1.13.6 — HTTP client for API communication
- Chart.js 4.5.1 — Data visualisation library
- react-chartjs-2 5.3.1 — React wrapper for Chart.js
- react-dropzone 15.0.0 — Drag-and-drop file upload component
- ESLint 9.39.4 — JavaScript linting tool

**Hardware Requirements:**

- Processor: Minimum Intel Core i5 (8th generation or equivalent) or AMD Ryzen 5; recommended Intel Core i7 or AMD Ryzen 7 for handling concurrent matching requests with NLP models
- RAM: Minimum 8 GB; recommended 16 GB, particularly when loading sentence-transformer models into memory
- Storage: Minimum 5 GB of free disk space for application files, Python environment, AI model weights, and uploaded resume storage
- Operating System: Windows 10/11, macOS 12+, or Ubuntu 20.04 LTS or later
- Network: Local network or internet connection required for initial package installation and CORS-enabled API communication between the frontend (port 5173) and backend (port 8000)
- Browser: Modern Chromium-based browser (Google Chrome 110+, Microsoft Edge 110+) or Firefox 110+ for the React frontend

### 3.2 Functional and Non-Functional Requirements

#### Functional Requirements

- **FR-01:** The system shall accept resume file uploads in PDF, DOCX, and TXT formats with a maximum file size of 10 MB per file.
- **FR-02:** The system shall parse uploaded resumes and extract the candidate's name, email address, phone number, skills (up to 30), education level, years of experience, project names, and certifications.
- **FR-03:** The system shall accept job description submissions via a web form containing a job title and free-form job description text.
- **FR-04:** The system shall parse job descriptions to extract required skills, preferred skills, minimum experience requirements, and required education level.
- **FR-05:** The system shall compute a match score between a resume and a job description using a configurable weighted formula across skill coverage, experience, project relevance, and education.
- **FR-06:** The system shall classify a candidate as "Selected" if their match score meets or exceeds the configured threshold (default: 60%), and as "Rejected" otherwise.
- **FR-07:** The system shall generate a detailed explanation for each screening decision, including matched skills, missing skills, feature contribution breakdowns, strengths, weaknesses, and a natural language summary.
- **FR-08:** The system shall support batch processing, where all uploaded resumes are matched against a single job description in a single operation.
- **FR-09:** The system shall rank candidates by match score in descending order and display per-rank statistics including total candidates, selected count, rejected count, and average score.
- **FR-10:** The system shall support side-by-side comparison of any two uploaded candidates for the same job, producing pros, cons, and a natural language recommendation.
- **FR-11:** The system shall persist all uploaded resumes, job descriptions, and screening results in a relational database and allow them to be retrieved in subsequent sessions.
- **FR-12:** The system shall allow deletion of resumes and job descriptions, with cascading deletion of associated screening results.

- **FR-13:** The system shall serve uploaded resume files as static assets accessible via the /uploads/ URL path.

**Non-Functional Requirements**

- **NFR-01 (Performance):** The system shall produce a screening result for a single resume-job pair within 5 seconds under normal operating conditions on the specified minimum hardware.
- **NFR-02 (Scalability):** The modular AI pipeline architecture shall allow individual modules (e.g., matching engine, explainability engine) to be replaced or upgraded independently without requiring changes to other modules.
- **NFR-03 (Usability):** The frontend shall implement a guided multi-step workflow (upload → job description → results) with clear visual progress indicators, reducing the need for user training.
- **NFR-04 (Reliability):** The system shall handle invalid or malformed resume files gracefully, returning meaningful HTTP 400 error messages rather than crashing, and storing partial parse results where possible.
- **NFR-05 (Security):** File uploads shall be validated for permitted extensions (.pdf, .docx, .txt) and size limits (10 MB) before processing, and stored with UUID-based filenames to prevent filename collision and directory traversal attacks.
- **NFR-06 (Maintainability):** All AI modules shall be implemented as independent Python packages with clearly defined interfaces and docstrings, enabling future developers to modify or extend individual components without requiring full system knowledge.
- **NFR-07 (Portability):** The backend shall be configurable via environment variables (e.g., DATABASE_URL) to support deployment in different environments without code changes.
- **NFR-08 (Transparency):** Every screening decision produced by the system shall be accompanied by a complete, human-readable explanation, such that no result is presented to the user without an accompanying justification.

### 3.3 Other Requirements

- **Data Storage Requirements:** Uploaded resume files are stored on the local filesystem under the data/resumes/ directory, with each file renamed to a UUID-based name upon

upload. The SQLite database file is stored at database/hirelens.db. In a production deployment, both the file storage and the database would require migration to cloud object storage (e.g., AWS S3) and a managed relational database (e.g., AWS RDS or Google Cloud SQL) respectively.

- **CORS Configuration:** The backend is configured to accept cross-origin requests from http://localhost:5173 and http://localhost:3000, corresponding to the default Vite development server ports. In production, this configuration must be updated to reflect the deployed frontend origin.
- **AI Model Dependencies:** The sentence-transformers library requires downloading pre-trained transformer model weights (approximately 90 MB for the default all-MiniLM-L6-v2 model) from the Hugging Face Hub on first use. This requires an active internet connection during the initial setup. The spaCy pipeline requires downloading a language model (e.g., en_core_web_sm) via the command python -m spacy download en_core_web_sm.
- **Compliance Requirements:** The system is designed with GDPR Article 22 compliance principles in mind through its explainability features. In a production deployment handling actual candidate data, additional measures would be required, including data retention policies, consent mechanisms, and data subject access request (DSAR) procedures.
- **Configuration Requirements:** Key system parameters are centralised in backend/core/config.py and include: the database URL, upload directory path, allowed file extensions, maximum file size, CORS origins, matching weights, and the selection threshold. Any modification to scoring behaviour (e.g., changing the skills weight from 40% to 50%) requires only a change to this single configuration file.

# CHAPTER 4
# SYSTEM DESIGN

## 4.1 Architecture Diagram

HireLens AI follows a classic three-tier web application architecture composed of a Presentation Tier (React frontend), an Application/Logic Tier (FastAPI backend + AI pipeline), and a Data Tier (SQLite database + filesystem). The three tiers communicate through a well-defined REST API interface and are kept cleanly separated, ensuring that each tier can be independently tested, scaled, or replaced.

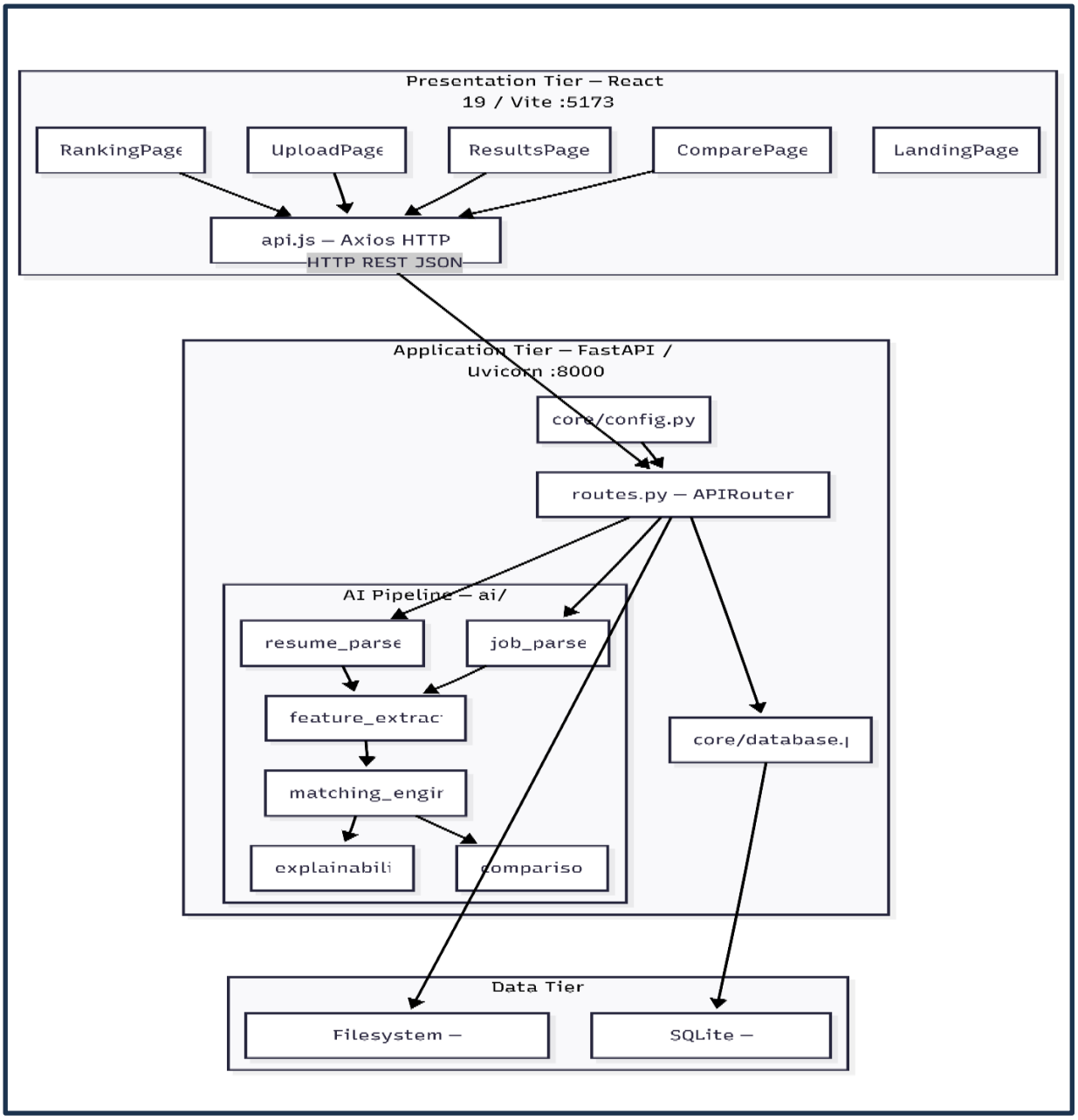


**Fig 4.1 System Architecture**

## 4.2 UML Diagrams

### Use Case Diagram

Two actors operate in HireLens AI: the **Recruiter** (human user) and the **AI System** (automated backend pipeline). The Recruiter initiates all actions; the AI System responds by executing analytical operations.

**Recruiter use cases:** Upload resume; submit job description; match single resume; batch-match all resumes; view screening results; view candidate rankings; compare two candidates; delete resume/job.

**AI System use cases (triggered internally):** Parse resume fields; parse job description; compute feature scores; aggregate weighted match score; apply selection threshold; generate explainability report; generate comparison and recommendation.

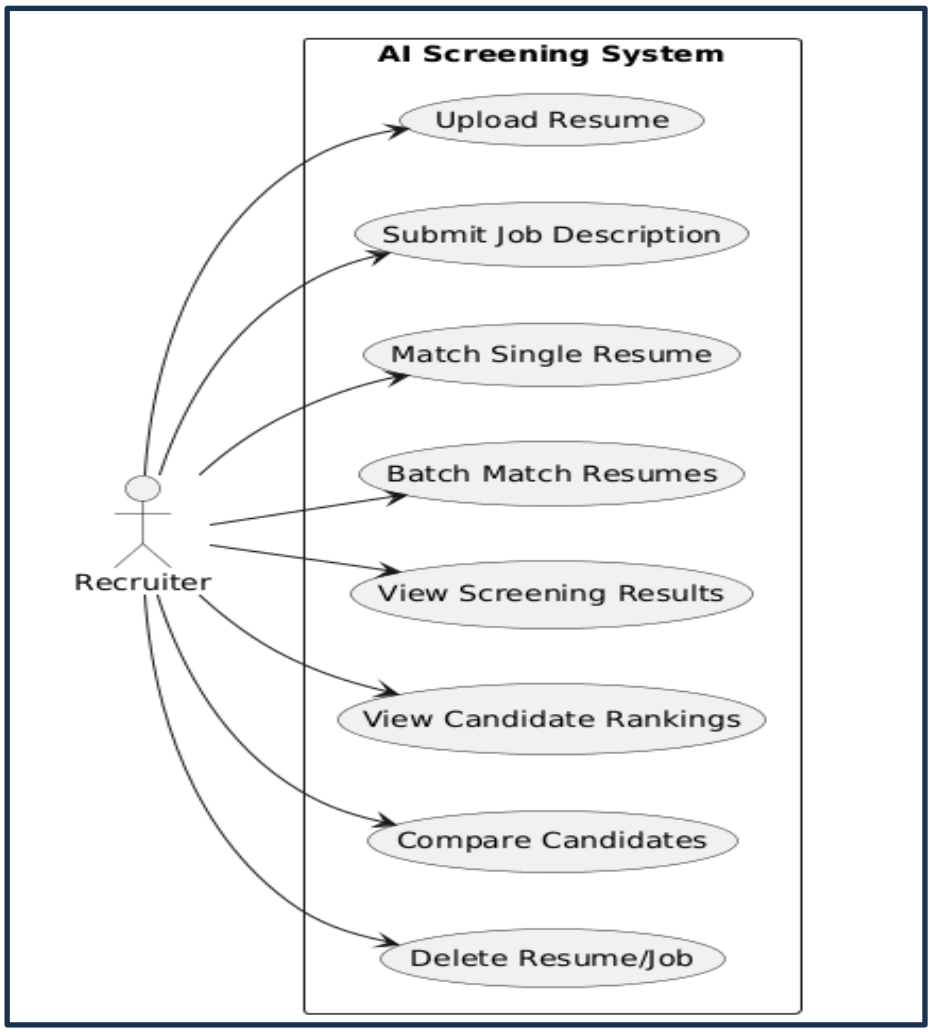


**Fig 4.2 Use Case Diagram**

## Class Diagram

The core classes of HireLens AI consist of three SQLAlchemy database models and six AI pipeline classes. The diagram below shows attributes, key methods, and inter-class dependencies.

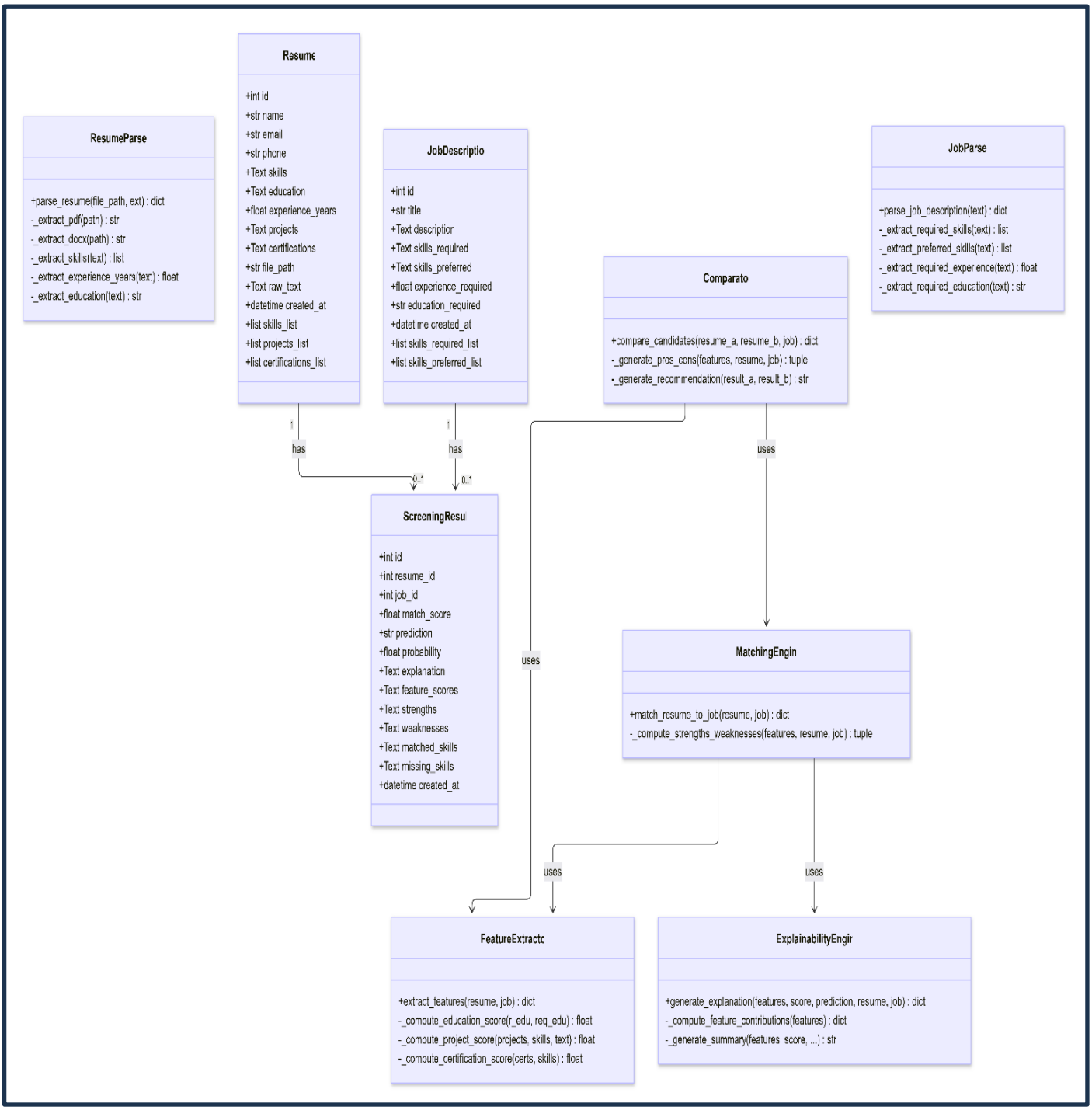


**Fig 4.3 Class Diagram**

## Sequence Diagram

The sequence diagram below illustrates the complete request-response flow for the primary use case: uploading a resume, submitting a job description, and receiving a screening result.

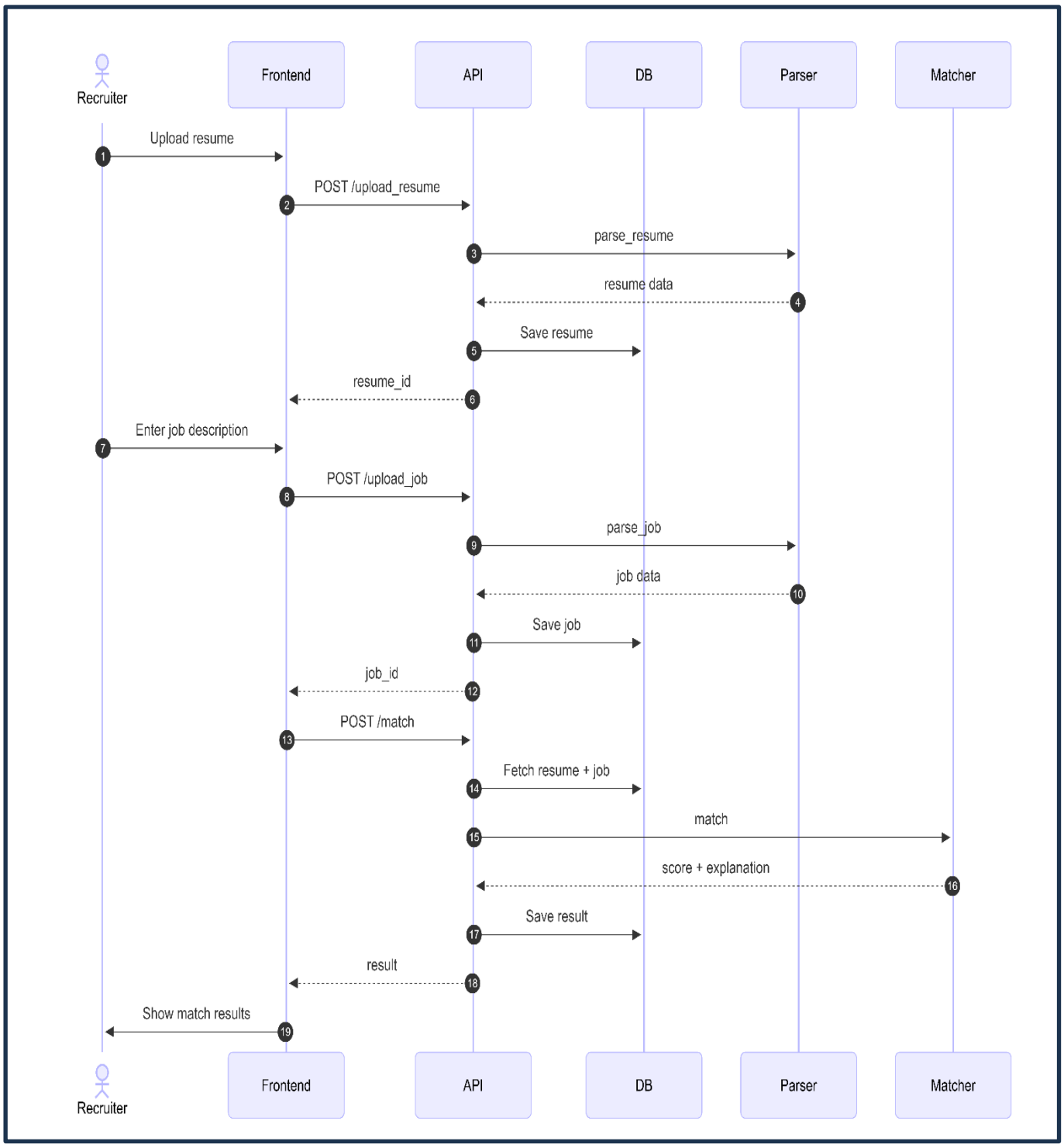


**Fig 4.4 Sequence Diagram**

## Activity Diagram

The activity diagram describes the complete operational workflow of the system, from the recruiter landing on the application to receiving ranked results or performing candidate comparisons.

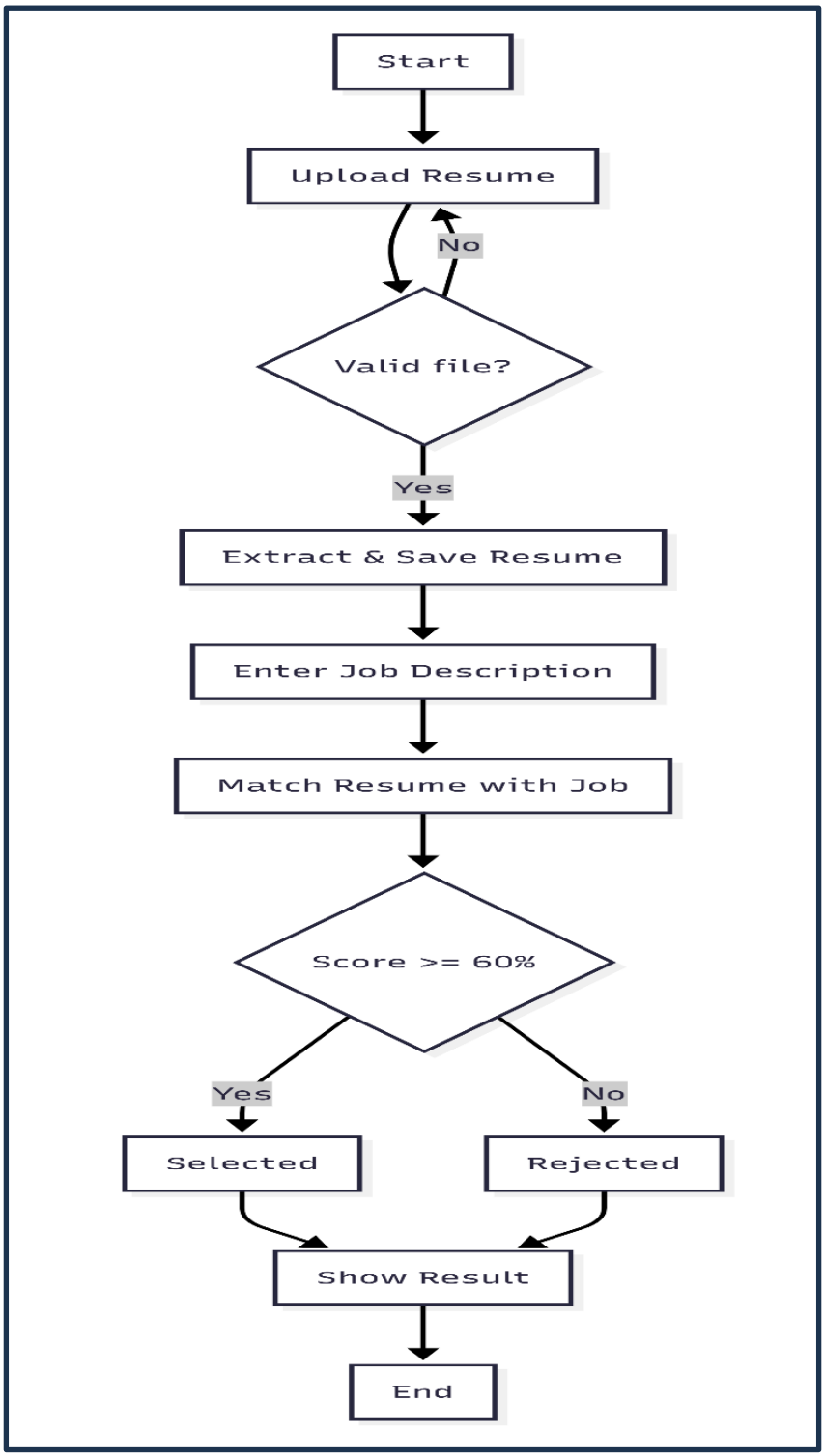


**Fig 4.5 Activity Diagram**

# CHAPTER 5
# IMPLEMENTATION

## 5.1 Algorithms

**1. Resume Text Extraction:**

The resume parser supports three document types, each requiring a distinct extraction strategy. For PDF files, the pdfplumber library iterates over all pages and concatenates the text extracted from each page. For DOCX files, python-docx reads all paragraphs and joins non-empty paragraph texts with newline delimiters. For TXT files, the file is read directly using UTF-8 encoding with error-ignore fallback. In all cases, the raw extracted text is returned as a single string for downstream processing.

**2. Skill Extraction via Ontology Matching :**

The skill extraction algorithm maintains a curated ontology (KNOWN_SKILLS) of over 150 technical terms spanning programming languages, web frameworks, AI/ML libraries, databases, cloud platforms, DevOps tools, testing frameworks, and mobile technologies. The algorithm converts the raw resume text to lowercase and iterates through the skill ontology sorted by term length in descending order, ensuring that multi-word skills (e.g., "machine learning", "spring boot") are matched before their constituent single words. For single-word skills, word-boundary anchors (\b) are applied to prevent false positives (e.g., matching "r" inside "recruiter"). Matched skills are capitalised and capped at 30 entries per resume. The same ontology and algorithm are reused in the job description parser.

**3. Feature Score Computation:**

Five independent feature scores are computed within feature_extractor.py, each normalised to the [0, 1] range:

- Skill Score: The set intersection of the candidate's skills and all job skills (required ∪ preferred), divided by the total number of job skills. If no job skills are specified, a neutral score of 0.5 is assigned.

- Experience Score: The ratio of the candidate's experience years to the required years, capped at 1.0. If no requirement is specified, a scaled score based on absolute years (÷3) is used.

- Education Score: An ordinal hierarchy (Diploma=1 < Bachelor's=2 < Master's=3 < PhD=4) is applied to both the candidate's and the job's education level. A score of 1.0 is assigned if the candidate meets or exceeds the requirement; 0.6 if one level below; 0.3 otherwise.

- Project Relevance Score: The number of job skill keywords present in the concatenated project descriptions, scaled by 30% of the total job skills count, capped at 1.0.

- Certification Score: The number of job skill keywords found in the concatenated certification text, normalised by the total job skills count, with a base score of 0.3, capped at 1.0.

**4. Weighted Score Aggregation and Prediction**

The final match score is computed by the matching engine as a weighted linear combination:

weighted_score = (0.40 × skill_score) + (0.25 × experience_score) + (0.20 × project_score) + (0.15 × education_score)
final_score = min(weighted_score + (certification_score × 0.05), 1.0)
match_percentage = round(final_score × 100, 1)
prediction = "Selected" if final_score >= 0.60 else "Rejected"

The weights (40/25/20/15) and threshold (0.60) are defined in backend/core/config.py and are independently configurable without code changes.

**5. Explainability Report Generation**

For each feature, the explainability engine computes a weighted contribution (score × weight), labels the impact as "positive" (score ≥ 0.5) or "negative" (score < 0.5), and generates a descriptive string categorising the score as "Excellent" (≥0.8), "Good" (≥0.6), "Moderate" (≥0.4), "Low" (≥0.2), or "Very Low" (<0.2). A natural language summary is constructed by concatenating sentences about matched skills, missing skills, experience level, the strongest feature, and the weakest feature. The explanation structure is stored as a JSON column in the ScreeningResult table and rendered on the frontend by the ExplanationPanel component.

**6. Candidate Comparison**

The comparator invokes match_resume_to_job() and extract_features() independently for both candidates against the same job. Pros and cons are generated by evaluating each feature score against qualitative thresholds (e.g., skill_score ≥ 0.7 → pro; < 0.5 → con). A natural language recommendation is generated based on the absolute difference in match scores: if the difference is less than 5 percentage points, both candidates are described as closely matched; otherwise, the higher-scoring candidate is recommended with their respective scores cited

## 5.2 Source Code

**Core Logic (The Brain)**

**parser.py:**

```
"""
Resume Parser Module
Extracts structured data from PDF, DOCX, and TXT resume files.
Uses regex patterns and heuristics for section detection.
"""

import re
from pathlib import Path

def parse_resume(file_path: str, ext: str = None) -> dict:
    """
    Parse a resume file and return structured data.

    Args:
        file_path: Path to the resume file
        ext: File extension (auto-detected if not provided)

    Returns:
        Dict with name, email, phone, skills, education, experience_years,
        projects, certifications, raw_text
    """
    if ext is None:
        ext = Path(file_path).suffix.lower()

    # Extract raw text
    if ext == ".pdf":
        raw_text = _extract_pdf(file_path)
```

```
    elif ext == ".docx":
        raw_text = _extract_docx(file_path)
    elif ext == ".txt":
        raw_text = _extract_txt(file_path)
    else:
        raw_text = _extract_txt(file_path)

    # Parse structured fields
    return {
        "name": _extract_name(raw_text),
        "email": _extract_email(raw_text),
        "phone": _extract_phone(raw_text),
        "skills": _extract_skills(raw_text),
        "education": _extract_education(raw_text),
        "experience_years": _extract_experience_years(raw_text),
        "projects": _extract_projects(raw_text),
        "certifications": _extract_certifications(raw_text),
        "raw_text": raw_text,
    }

# ─── Text Extraction ──────────────────────────────────────────────────────

def _extract_pdf(file_path: str) -> str:
    """Extract text from PDF using pdfplumber."""
    try:
        import pdfplumber
        text = ""
        with pdfplumber.open(file_path) as pdf:
            for page in pdf.pages:
                page_text = page.extract_text()
                if page_text:
                    text += page_text + "\n"
        return text.strip()
    except Exception as e:
        print(f"PDF extraction error: {e}")
        return ""

def _extract_docx(file_path: str) -> str:
    """Extract text from DOCX using python-docx."""
    try:
        from docx import Document
```

```
        doc = Document(file_path)
        text = "\n".join([para.text for para in doc.paragraphs if para.text.strip()])
        return text.strip()
    except Exception as e:
        print(f"DOCX extraction error: {e}")
        return ""

def _extract_txt(file_path: str) -> str:
    """Read plain text file."""
    try:
        with open(file_path, "r", encoding="utf-8", errors="ignore") as f:
            return f.read().strip()
    except Exception as e:
        print(f"TXT extraction error: {e}")
        return ""

# ─── Field Extraction ──────────────────────────────────────────────────────

def _extract_name(text: str) -> str:
    """Extract candidate name from resume text (first non-empty line heuristic)."""
    lines = [l.strip() for l in text.split("\n") if l.strip()]
    if not lines:
        return "Unknown"

    # The name is usually the first line of a resume
    first_line = lines[0]

    # Skip if it looks like a section header
    headers = ["resume", "curriculum vitae", "cv", "objective", "summary", "profile"]
    if first_line.lower() in headers:
        if len(lines) > 1:
            return lines[1][:80]

    # Skip if first line contains an email or phone
    if "@" in first_line or re.search(r"\d{10}", first_line):
        if len(lines) > 1:
            return lines[1][:80]

    return first_line[:80]

def _extract_email(text: str) -> str:
    """Extract email address."""
```

```
    match = re.search(r"[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}", text)
    return match.group(0) if match else None

def _extract_phone(text: str) -> str:
    """Extract phone number."""
    patterns = [
        r"\+?\d{1,3}[-.\s]?\(?\d{2,4}\)?[-.\s]?\d{3,4}[-.\s]?\d{3,4}",
        r"\(?\d{3}\)?[-.\s]?\d{3}[-.\s]?\d{4}",
        r"\d{10,12}",
    ]
    for pattern in patterns:
        match = re.search(pattern, text)
        if match:
            return match.group(0).strip()
    return None

# ─── Skills Extraction ──────────────────────────────────────────────────────

# Comprehensive skill list for matching
KNOWN_SKILLS = {
    # Programming languages
    "python", "java", "javascript", "typescript", "c++", "c#", "c", "ruby", "go",
    "golang", "rust", "swift", "kotlin", "php", "scala", "r", "matlab", "perl",
    "dart", "lua", "haskell", "elixir", "clojure", "groovy",
    # Web frameworks
    "react", "reactjs", "react.js", "angular", "angularjs", "vue", "vuejs", "vue.js",
    "django", "flask", "fastapi", "express", "expressjs", "next.js", "nextjs",
    "nuxt.js", "nuxtjs", "spring", "spring boot", "rails", "ruby on rails",
    "asp.net", "laravel", "svelte", "gatsby",
    # Data & AI/ML
    "machine learning", "deep learning", "artificial intelligence", "ai", "ml",
    "natural language processing", "nlp", "computer vision", "tensorflow", "pytorch",
    "keras", "scikit-learn", "sklearn", "pandas", "numpy", "scipy", "matplotlib",
    "seaborn", "opencv", "spacy", "nltk", "hugging face", "transformers",
    "data science", "data analysis", "data engineering", "data visualization",
    "big data", "statistics", "neural networks",
    # Databases
    "sql", "mysql", "postgresql", "postgres", "mongodb", "redis", "sqlite",
    "oracle", "cassandra", "dynamodb", "elasticsearch", "neo4j", "firebase",
    "mariadb", "couchdb",
    # Cloud & DevOps
    "aws", "amazon web services", "azure", "gcp", "google cloud", "docker",
```

```
    "kubernetes", "k8s", "jenkins", "ci/cd", "terraform", "ansible", "linux",
    "unix", "git", "github", "gitlab", "bitbucket", "nginx", "apache",
    "heroku", "vercel", "netlify", "cloudflare",
    # Tools & Technologies
    "rest api", "rest apis", "graphql", "websocket", "microservices",
    "api", "json", "xml", "html", "css", "sass", "less", "bootstrap",
    "tailwind", "tailwind css", "webpack", "babel", "npm", "yarn",
    "agile", "scrum", "jira", "confluence", "trello",
    "power bi", "tableau", "excel", "word",
    # Mobile
    "android", "ios", "react native", "flutter", "xamarin", "ionic",
    # Testing
    "unit testing", "integration testing", "selenium", "jest", "mocha",
    "pytest", "junit", "cypress", "playwright",
    # Other
    "blockchain", "iot", "embedded systems", "robotics", "ar", "vr",
    "game development", "unity", "unreal engine",
}

def _extract_skills(text: str) -> list:
    """Extract skills from resume text by matching known skills."""
    text_lower = text.lower()
    found_skills = []

    # Sort skills by length (longest first) to match multi-word skills first
    sorted_skills = sorted(KNOWN_SKILLS, key=len, reverse=True)

    for skill in sorted_skills:
        # Use word boundary matching for single-word skills
        if len(skill.split()) == 1:
            pattern = r"\b" + re.escape(skill) + r"\b"
        else:
            pattern = re.escape(skill)

        if re.search(pattern, text_lower):
            # Capitalize properly
            display_skill = skill.title() if len(skill) > 3 else skill.upper()
            if display_skill not in found_skills:
                found_skills.append(display_skill)

    return found_skills[:30]  # Cap at 30 skills
```

```
def _extract_education(text: str) -> str:
    """Extract education information."""
    education_patterns = [
        r"(?:B\.?Tech|B\.?E\.?|Bachelor(?:'s)?|B\.?Sc\.?|B\.?A\.?|B\.?S\.?)[\s\w,.-]+",
        r"(?:M\.?Tech|M\.?E\.?|Master(?:'s)?|M\.?Sc\.?|M\.?A\.?|M\.?S\.?|MBA)[\s\w,.-]+",
        r"(?:Ph\.?D\.?|Doctorate|Doctor)[\s\w,.-]+",
        r"(?:Diploma|Associate(?:'s)?)[\s\w,.-]+",
    ]

    for pattern in education_patterns:
        match = re.search(pattern, text, re.IGNORECASE)
        if match:
            return match.group(0).strip()[:200]

    # Try to find education section
    edu_section = _extract_section(text, ["education", "academic", "qualification"])
    if edu_section:
        return edu_section[:200]

    return None

def _extract_experience_years(text: str) -> float:
    """Extract years of experience."""
    patterns = [
        r"(\d+)\+?\s*(?:years?|yrs?)\s*(?:of)?\s*(?:experience|exp)",
        r"(?:experience|exp)[\s:]*(\d+)\+?\s*(?:years?|yrs?)",
        r"(\d+)\+?\s*(?:years?|yrs?)\s*(?:in|of)\s*(?:industry|work|professional)",
    ]

    for pattern in patterns:
        match = re.search(pattern, text, re.IGNORECASE)
        if match:
            return float(match.group(1))

    # Count work experience entries to estimate
    work_entries = len(re.findall(
        r"(?:20\d{2}|19\d{2})\s*[-——to]+\s*(?:20\d{2}|19\d{2}|present|current)",
        text, re.IGNORECASE
    ))
    if work_entries > 0:
        return float(work_entries)  # Rough estimate
```

```
    return 0

def _extract_projects(text: str) -> list:
    """Extract project names/descriptions."""
    project_section = _extract_section(text, ["projects", "project", "personal projects",
"academic projects"])
    if not project_section:
        return []

    lines = [l.strip() for l in project_section.split("\n") if l.strip()]
    projects = []
    for line in lines[:10]:
        # Clean up line
        line = re.sub(r"^[-•*▪►◆→|]\s*", "", line).strip()
        if line and len(line) > 3 and len(line) < 200:
            projects.append(line)

    return projects[:10]

def _extract_certifications(text: str) -> list:
    """Extract certifications."""
    cert_section = _extract_section(
        text, ["certifications", "certification", "certificates", "licenses", "credentials"]
    )
    if not cert_section:
        # Also look for common certification keywords
        cert_patterns = [
            r"(?:AWS|Azure|GCP|Google|Cisco|Oracle|Microsoft|CompTIA)\s*[\w\s-
]+(?:cert\w*|practitioner|associate|professional|expert|specialist)",
            r"(?:PMP|CISSP|CCNA|CCNP|CKA|CKAD|CKS)\b",
        ]
        found = []
        for pattern in cert_patterns:
            matches = re.findall(pattern, text, re.IGNORECASE)
            found.extend(matches)
        return [m.strip() for m in found[:10]]

    lines = [l.strip() for l in cert_section.split("\n") if l.strip()]
    certs = []
    for line in lines[:10]:
        line = re.sub(r"^[-•*▪►◆→|]\s*", "", line).strip()
        if line and len(line) > 3 and len(line) < 200:
```

```
            certs.append(line)

    return certs[:10]

def _extract_section(text: str, section_headers: list) -> str:
    """Extract text content under a given section header."""
    text_lines = text.split("\n")
    capturing = False
    content = []

    # Common section headers that indicate end of current section
    all_headers = {
        "education", "experience", "skills", "projects", "certifications",
        "work experience", "professional experience", "technical skills",
        "personal projects", "academic projects", "achievements", "awards",
        "publications", "references", "interests", "hobbies", "summary",
        "objective", "profile", "about", "contact", "languages",
        "qualifications", "training", "courses", "volunteer",
    }

    for line in text_lines:
        stripped = line.strip().lower().rstrip(":")
        stripped = re.sub(r"^[#=\-_*]+\s*", "", stripped).strip()

        if any(h == stripped for h in section_headers):
            capturing = True
            continue

        if capturing:
            # Check if we hit another section header
            if stripped in all_headers and stripped not in section_headers:
                break
            content.append(line)

    return "\n".join(content).strip() if content else None
```

**matching.py:**

```
"""
Matching Engine Module
Calculates resume-job similarity using feature scoring and weighted combination.
Produces match score, prediction, and detailed explanation.
```

```
"""

import json
from ai.feature_extractor.extractor import extract_features
from ai.explainability.explainer import generate_explanation
from backend.core.config import MATCHING_WEIGHTS, SELECTION_THRESHOLD

def match_resume_to_job(resume, job) -> dict:
    """
    Match a resume against a job description.

    Args:
        resume: Resume DB model instance
        job: JobDescription DB model instance

    Returns:
        Dict with match_score, prediction, probability, feature_scores,
        explanation, strengths, weaknesses, matched_skills, missing_skills
    """
    # Step 1: Extract features
    features = extract_features(resume, job)

    # Step 2: Compute weighted score
    weights = MATCHING_WEIGHTS
    weighted_score = (
        weights["skill_score"] * features["skill_score"] +
        weights["experience_score"] * features["experience_score"] +
        weights["project_score"] * features["project_score"] +
        weights["education_score"] * features["education_score"]
    )

    # Bonus for certifications (up to 5% boost)
    cert_bonus = features.get("certification_score", 0) * 0.05
    final_score = min(weighted_score + cert_bonus, 1.0)
    match_percentage = round(final_score * 100, 1)

    # Step 3: Prediction
    threshold = SELECTION_THRESHOLD
    prediction = "Selected" if final_score >= threshold else "Rejected"
    probability = round(final_score, 3)

    # Step 4: Generate explanation
```

```
    explanation = generate_explanation(
        features=features,
        match_score=match_percentage,
        prediction=prediction,
        resume=resume,
        job=job,
    )

    # Step 5: Compute strengths and weaknesses
    strengths, weaknesses = _compute_strengths_weaknesses(features, resume, job)

    return {
        "match_score": match_percentage,
        "prediction": prediction,
        "probability": probability,
        "feature_scores": {
            "skill_score": features["skill_score"],
            "experience_score": features["experience_score"],
            "education_score": features["education_score"],
            "project_score": features["project_score"],
            "certification_score": features.get("certification_score", 0),
            "semantic_score": 0.0,  # Placeholder for semantic scoring
        },
        "explanation": explanation,
        "strengths": strengths,
        "weaknesses": weaknesses,
        "matched_skills": features.get("matched_skills", []),
        "missing_skills": features.get("missing_skills", []),
    }

def _compute_strengths_weaknesses(features: dict, resume, job) -> tuple:
    """Generate human-readable strengths and weaknesses."""
    strengths = []
    weaknesses = []

    # Skill analysis
    skill_score = features["skill_score"]
    matched = features.get("matched_skills", [])
    missing = features.get("missing_skills", [])

    if skill_score >= 0.8:
        strengths.append(f"Strong skill coverage ({len(matched)} matched skills)")
```

```
    elif skill_score >= 0.5:
        strengths.append(f"Moderate skill coverage ({len(matched)} matched)")
    else:
        weaknesses.append(f"Low skill coverage (only {len(matched)} matched)")

    if matched:
        top_skills = ", ".join(matched[:5])
        strengths.append(f"Key skills matched: {top_skills}")

    if missing:
        top_missing = ", ".join(missing[:5])
        weaknesses.append(f"Missing skills: {top_missing}")

    # Experience analysis
    exp_score = features["experience_score"]
    resume_exp = resume.experience_years or 0
    required_exp = job.experience_required or 0

    if exp_score >= 0.9:
        strengths.append(f"Meets experience requirement ({resume_exp}+ years)")
    elif exp_score >= 0.6:
        strengths.append(f"Partially meets experience ({resume_exp} years vs {required_exp}
required)")
    elif required_exp > 0:
        weaknesses.append(f"Insufficient experience ({resume_exp} years vs {required_exp}
required)")

    # Education analysis
    edu_score = features["education_score"]
    if edu_score >= 0.8:
        strengths.append("Education requirements met")
    elif edu_score < 0.5:
        weaknesses.append("Education level does not meet requirements")

    # Project analysis
    proj_score = features["project_score"]
    if proj_score >= 0.6:
        strengths.append("Relevant project experience")
    elif proj_score < 0.3:
        weaknesses.append("Limited relevant project experience")

    # Certification analysis
```

```
    cert_score = features.get("certification_score", 0)
    if cert_score >= 0.5:
        strengths.append("Relevant certifications present")
    elif cert_score < 0.2:
        weaknesses.append("No relevant certifications")

    return strengths, weaknesses
```

**explainer.py:**

```
"""
Explainability Engine Module
Generates human-readable explanations for AI screening decisions.
Implements feature contribution analysis and natural language summaries.
"""

import json

def generate_explanation(features: dict, match_score: float, prediction: str,
                         resume, job) -> dict:
    """
    Generate a comprehensive explanation for the screening decision.

    Args:
        features: Feature scores dict from feature extractor
        match_score: Final match percentage
        prediction: "Selected" or "Rejected"
        resume: Resume DB model instance
        job: JobDescription DB model instance

    Returns:
        Dict with matched_skills, missing_skills, strengths, weaknesses,
        feature_contributions, summary
    """
    matched_skills = features.get("matched_skills", [])
    missing_skills = features.get("missing_skills", [])

    # Feature contributions (how much each feature helped/hurt)
    feature_contributions = _compute_feature_contributions(features)

    # Generate human-readable summary
    summary = _generate_summary(
```

```
    features=features,
    match_score=match_score,
    prediction=prediction,
    matched_skills=matched_skills,
    missing_skills=missing_skills,
    resume=resume,
    job=job,
)

# Strengths and weaknesses from features
strengths = []
weaknesses = []

for skill in matched_skills[:5]:
    strengths.append(f"+ {skill} skill matched")

if features["experience_score"] >= 0.7:
    strengths.append("+ Sufficient work experience")
elif features["experience_score"] < 0.4:
    weaknesses.append("- Insufficient experience")

if features["education_score"] >= 0.7:
    strengths.append("+ Education requirement met")
elif features["education_score"] < 0.4:
    weaknesses.append("- Education gap")

if features["project_score"] >= 0.5:
    strengths.append("+ Relevant project experience found")
else:
    weaknesses.append("- Limited project relevance")

for skill in missing_skills[:5]:
    weaknesses.append(f"- Missing {skill}")

return {
    "matched_skills": matched_skills,
    "missing_skills": missing_skills,
    "strengths": strengths,
    "weaknesses": weaknesses,
    "feature_contributions": feature_contributions,
    "summary": summary,
}
```

```
def _compute_feature_contributions(features: dict) -> dict:
    """
    Compute each feature's contribution to the final score.
    Returns a dict mapping feature name → contribution description.
    """
    contributions = {}

    feature_labels = {
        "skill_score": ("Skills Match", 0.40),
        "experience_score": ("Experience Match", 0.25),
        "project_score": ("Project Relevance", 0.20),
        "education_score": ("Education Match", 0.15),
    }

    for key, (label, weight) in feature_labels.items():
        score = features.get(key, 0)
        weighted = score * weight
        impact = "positive" if score >= 0.5 else "negative"
        contributions[label] = {
            "score": round(score, 3),
            "weight": weight,
            "weighted_contribution": round(weighted, 3),
            "impact": impact,
            "description": _describe_score(label, score),
        }

    return contributions

def _describe_score(feature_name: str, score: float) -> str:
    """Generate a human-readable description for a feature score."""
    if score >= 0.8:
        quality = "Excellent"
    elif score >= 0.6:
        quality = "Good"
    elif score >= 0.4:
        quality = "Moderate"
    elif score >= 0.2:
        quality = "Low"
    else:
        quality = "Very Low"
```

```
    return f"{quality} {feature_name.lower()} ({round(score * 100)}%)"

def _generate_summary(features, match_score, prediction, matched_skills,
                      missing_skills, resume, job) -> str:
    """Generate a natural language summary of the screening decision."""
    candidate_name = resume.name or "The candidate"
    job_title = job.title or "the position"

    lines = []
    lines.append(
        f"{candidate_name} received a match score of {match_score}% "
        f"for {job_title} and is predicted to be {prediction}."
    )

    # Skills summary
    if matched_skills:
        top = ", ".join(matched_skills[:5])
        lines.append(f"Matched skills include: {top}.")

    if missing_skills:
        top = ", ".join(missing_skills[:5])
        lines.append(f"Missing required skills: {top}.")

    # Experience summary
    exp_score = features["experience_score"]
    resume_exp = resume.experience_years or 0
    if exp_score >= 0.7:
        lines.append(f"The candidate has {resume_exp} years of experience, meeting the job
requirements.")
    elif resume_exp > 0:
        lines.append(f"The candidate has {resume_exp} years of experience, which partially
meets the requirements.")
    else:
        lines.append("No specific experience duration was identified from the resume.")

    # Key factor
    scores = {
        "Skills": features["skill_score"],
        "Experience": features["experience_score"],
        "Projects": features["project_score"],
        "Education": features["education_score"],
    }
```

```
    top_factor = max(scores, key=scores.get)
    bottom_factor = min(scores, key=scores.get)

    lines.append(f"The strongest factor is {top_factor} ({round(scores[top_factor]*100)}%).")
    if scores[bottom_factor] < 0.5:
        lines.append(f"The weakest area is {bottom_factor}
({round(scores[bottom_factor]*100)}%).")

    return " ".join(lines)
```

**Backend Logic (The Server & API)**

**main.py:**

```
import sys
from pathlib import Path

# Add project root to sys.path
sys.path.insert(0, str(Path(__file__).resolve().parent.parent))

from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from fastapi.staticfiles import StaticFiles

from backend.core.config import CORS_ORIGINS, UPLOAD_DIR
from backend.core.database import init_db
from backend.api.routes import router

app = FastAPI(
    title="HireLens AI",
    description="Explainable AI Resume Screening System — See exactly why candidates get
selected.",
    version="1.0.0",
)

# CORS
app.add_middleware(
    CORSMiddleware,
    allow_origins=CORS_ORIGINS,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
```

```
)

# Static file serving for uploaded resumes
UPLOAD_DIR.mkdir(parents=True, exist_ok=True)
app.mount("/uploads", StaticFiles(directory=str(UPLOAD_DIR)), name="uploads")

# Register routes
app.include_router(router)

@app.on_event("startup")
def startup():
    """Initialize database tables on startup."""
    init_db()

if __name__ == "__main__":
    import uvicorn
    uvicorn.run("backend.main:app", host="0.0.0.0", port=8000, reload=True)
```

**routes.py**

```
import json
import os
import uuid
from pathlib import Path
from typing import List

from fastapi import APIRouter, Depends, File, UploadFile, HTTPException, Form
from sqlalchemy.orm import Session

from backend.core.config import UPLOAD_DIR, ALLOWED_EXTENSIONS,
MAX_FILE_SIZE
from backend.core.database import get_db
from backend.models.models import Resume, JobDescription, ScreeningResult
from backend.schemas.schemas import (
    ResumeResponse,
    ResumeListResponse,
    JobDescriptionCreate,
    JobDescriptionResponse,
    JobListResponse,
    MatchRequest,
    MatchBatchRequest,
    ScreeningResultResponse,
```

```
    FeatureScores,
    ExplanationDetail,
    RankingResponse,
    RankingListResponse,
    CompareRequest,
    CompareResponse,
    CandidateComparison,
)

router = APIRouter(prefix="/api", tags=["HireLens AI"])

# ─── Resume Endpoints ──────────────────────────────────────────────────────

@router.post("/upload_resume", response_model=ResumeResponse)
async def upload_resume(file: UploadFile = File(...), db: Session = Depends(get_db)):
    """Upload and parse a resume file (PDF, DOCX, or TXT)."""
    # Validate extension
    ext = Path(file.filename).suffix.lower()
    if ext not in ALLOWED_EXTENSIONS:
        raise HTTPException(400, f"Unsupported file type: {ext}. Allowed:
{ALLOWED_EXTENSIONS}")

    # Read file
    content = await file.read()
    if len(content) > MAX_FILE_SIZE:
        raise HTTPException(400, f"File too large. Max {MAX_FILE_SIZE // (1024*1024)}
MB.")

    # Save file
    unique_name = f"{uuid.uuid4().hex}{ext}"
    file_path = UPLOAD_DIR / unique_name
    with open(file_path, "wb") as f:
        f.write(content)

    # Parse resume
    from ai.resume_parser.parser import parse_resume
    parsed = parse_resume(str(file_path), ext)

    # Store in DB
    resume = Resume(
        name=parsed.get("name", "Unknown"),
```

```
        email=parsed.get("email"),
        phone=parsed.get("phone"),
        skills=json.dumps(parsed.get("skills", [])),
        education=parsed.get("education"),
        experience_years=parsed.get("experience_years", 0),
        projects=json.dumps(parsed.get("projects", [])),
        certifications=json.dumps(parsed.get("certifications", [])),
        file_path=str(file_path),
        file_name=file.filename,
        raw_text=parsed.get("raw_text", ""),
    )
    db.add(resume)
    db.commit()
    db.refresh(resume)

    return ResumeResponse(
        id=resume.id,
        name=resume.name,
        email=resume.email,
        phone=resume.phone,
        skills=resume.skills_list,
        education=resume.education,
        experience_years=resume.experience_years,
        projects=resume.projects_list,
        certifications=resume.certifications_list,
        file_name=resume.file_name,
        created_at=resume.created_at,
    )

@router.get("/resumes", response_model=ResumeListResponse)
def list_resumes(db: Session = Depends(get_db)):
    """List all uploaded resumes."""
    resumes = db.query(Resume).order_by(Resume.created_at.desc()).all()
    items = []
    for r in resumes:
        items.append(ResumeResponse(
            id=r.id, name=r.name, email=r.email, phone=r.phone,
            skills=r.skills_list, education=r.education,
            experience_years=r.experience_years, projects=r.projects_list,
            certifications=r.certifications_list, file_name=r.file_name,
            created_at=r.created_at,
        ))
```

```
    return ResumeListResponse(resumes=items, total=len(items))

@router.delete("/resumes/{resume_id}")
def delete_resume(resume_id: int, db: Session = Depends(get_db)):
    """Delete a resume and its screening results."""
    resume = db.query(Resume).filter(Resume.id == resume_id).first()
    if not resume:
        raise HTTPException(404, "Resume not found")
    # Delete associated results
    db.query(ScreeningResult).filter(ScreeningResult.resume_id == resume_id).delete()
    # Delete file
    if resume.file_path and os.path.exists(resume.file_path):
        os.remove(resume.file_path)
    db.delete(resume)
    db.commit()
    return {"message": "Resume deleted successfully"}

# ─── Job Description Endpoints ──────────────────────────────────────────

@router.post("/upload_job", response_model=JobDescriptionResponse)
def upload_job(payload: JobDescriptionCreate, db: Session = Depends(get_db)):
    """Submit a job description for analysis."""
    from ai.job_parser.parser import parse_job_description
    parsed = parse_job_description(payload.description)

    job = JobDescription(
        title=payload.title,
        description=payload.description,
        skills_required=json.dumps(parsed.get("skills_required", [])),
        skills_preferred=json.dumps(parsed.get("skills_preferred", [])),
        experience_required=parsed.get("experience_required", 0),
        education_required=parsed.get("education_required"),
    )
    db.add(job)
    db.commit()
    db.refresh(job)

    return JobDescriptionResponse(
        id=job.id, title=job.title, description=job.description,
        skills_required=job.skills_required_list,
        skills_preferred=job.skills_preferred_list,
        experience_required=job.experience_required,
```

```
        education_required=job.education_required,
        created_at=job.created_at,
    )

@router.get("/jobs", response_model=JobListResponse)
def list_jobs(db: Session = Depends(get_db)):
    """List all job descriptions."""
    jobs = db.query(JobDescription).order_by(JobDescription.created_at.desc()).all()
    items = []
    for j in jobs:
        items.append(JobDescriptionResponse(
            id=j.id, title=j.title, description=j.description,
            skills_required=j.skills_required_list,
            skills_preferred=j.skills_preferred_list,
            experience_required=j.experience_required,
            education_required=j.education_required,
            created_at=j.created_at,
        ))
    return JobListResponse(jobs=items, total=len(items))

@router.delete("/jobs/{job_id}")
def delete_job(job_id: int, db: Session = Depends(get_db)):
    """Delete a job description and its screening results."""
    job = db.query(JobDescription).filter(JobDescription.id == job_id).first()
    if not job:
        raise HTTPException(404, "Job not found")
    db.query(ScreeningResult).filter(ScreeningResult.job_id == job_id).delete()
    db.delete(job)
    db.commit()
    return {"message": "Job description deleted successfully"}

# ─── Matching Endpoints ──────────────────────────────────────────────────────

@router.post("/match_resume", response_model=ScreeningResultResponse)
def match_resume(payload: MatchRequest, db: Session = Depends(get_db)):
    """Match a single resume against a job description."""
    resume = db.query(Resume).filter(Resume.id == payload.resume_id).first()
    if not resume:
        raise HTTPException(404, "Resume not found")

    job = db.query(JobDescription).filter(JobDescription.id == payload.job_id).first()
    if not job:
```

```
        raise HTTPException(404, "Job description not found")

    # Run matching pipeline
    from ai.matching_engine.matching import match_resume_to_job
    result = match_resume_to_job(resume, job)

    # Store result
    screening = ScreeningResult(
        resume_id=resume.id,
        job_id=job.id,
        match_score=result["match_score"],
        prediction=result["prediction"],
        probability=result["probability"],
        explanation=json.dumps(result["explanation"]),
        feature_scores=json.dumps(result["feature_scores"]),
        strengths=json.dumps(result["strengths"]),
        weaknesses=json.dumps(result["weaknesses"]),
        matched_skills=json.dumps(result["matched_skills"]),
        missing_skills=json.dumps(result["missing_skills"]),
    )
    db.add(screening)
    db.commit()
    db.refresh(screening)

    return _build_screening_response(screening, resume.name)

@router.post("/match_batch", response_model=List[ScreeningResultResponse])
def match_batch(payload: MatchBatchRequest, db: Session = Depends(get_db)):
    """Match multiple resumes against a job description."""
    job = db.query(JobDescription).filter(JobDescription.id == payload.job_id).first()
    if not job:
        raise HTTPException(404, "Job description not found")

    from ai.matching_engine.matching import match_resume_to_job

    results = []
    for rid in payload.resume_ids:
        resume = db.query(Resume).filter(Resume.id == rid).first()
        if not resume:
            continue

        result = match_resume_to_job(resume, job)
```

```
        screening = ScreeningResult(
            resume_id=resume.id,
            job_id=job.id,
            match_score=result["match_score"],
            prediction=result["prediction"],
            probability=result["probability"],
            explanation=json.dumps(result["explanation"]),
            feature_scores=json.dumps(result["feature_scores"]),
            strengths=json.dumps(result["strengths"]),
            weaknesses=json.dumps(result["weaknesses"]),
            matched_skills=json.dumps(result["matched_skills"]),
            missing_skills=json.dumps(result["missing_skills"]),
        )
        db.add(screening)
        db.commit()
        db.refresh(screening)

        results.append(_build_screening_response(screening, resume.name))

    return results

# ─── Results & Ranking ──────────────────────────────────────────────────────


@router.get("/results/{job_id}", response_model=RankingListResponse)
def get_results(job_id: int, db: Session = Depends(get_db)):
    """Get all screening results for a job, ranked by score."""
    job = db.query(JobDescription).filter(JobDescription.id == job_id).first()
    if not job:
        raise HTTPException(404, "Job description not found")

    screenings = (
        db.query(ScreeningResult)
        .filter(ScreeningResult.job_id == job_id)
        .order_by(ScreeningResult.match_score.desc())
        .all()
    )

    rankings = []
    for rank, s in enumerate(screenings, start=1):
        resume = db.query(Resume).filter(Resume.id == s.resume_id).first()
```

```
        rankings.append(RankingResponse(
            rank=rank,
            resume_id=s.resume_id,
            candidate_name=resume.name if resume else "Unknown",
            match_score=s.match_score,
            prediction=s.prediction,
        ))

    return RankingListResponse(job_id=job_id, job_title=job.title, rankings=rankings)

@router.get("/results/{job_id}/detail/{resume_id}",
response_model=ScreeningResultResponse)
def get_result_detail(job_id: int, resume_id: int, db: Session = Depends(get_db)):
    """Get detailed screening result for a specific resume-job pair."""
    screening = (
        db.query(ScreeningResult)
        .filter(ScreeningResult.job_id == job_id, ScreeningResult.resume_id == resume_id)
        .order_by(ScreeningResult.created_at.desc())
        .first()
    )
    if not screening:
        raise HTTPException(404, "Screening result not found")

    resume = db.query(Resume).filter(Resume.id == resume_id).first()
    return _build_screening_response(screening, resume.name if resume else "Unknown")

# ─── Comparison Endpoint ──────────────────────────────────────────────────

@router.post("/compare_resumes", response_model=CompareResponse)
def compare_resumes(payload: CompareRequest, db: Session = Depends(get_db)):
    """Compare two resumes against a job description."""
    resume_a = db.query(Resume).filter(Resume.id == payload.resume_id_a).first()
    resume_b = db.query(Resume).filter(Resume.id == payload.resume_id_b).first()
    job = db.query(JobDescription).filter(JobDescription.id == payload.job_id).first()

    if not resume_a or not resume_b:
        raise HTTPException(404, "One or both resumes not found")
    if not job:
        raise HTTPException(404, "Job description not found")

    from ai.comparison.comparator import compare_candidates
    comparison = compare_candidates(resume_a, resume_b, job)
```

```
    return CompareResponse(
        job_id=job.id,
        job_title=job.title,
        candidate_a=CandidateComparison(**comparison["candidate_a"]),
        candidate_b=CandidateComparison(**comparison["candidate_b"]),
        recommendation=comparison["recommendation"],
    )

# ─── Health Check ──────────────────────────────────────────────────────────────

@router.get("/health")
def health_check():
    return {"status": "healthy", "service": "HireLens AI"}

# ─── Helpers ───────────────────────────────────────────────────────────────────

def _build_screening_response(screening: ScreeningResult, candidate_name: str) -> ScreeningResultResponse:
    """Convert a ScreeningResult DB row into a response schema."""
    feature_scores = json.loads(screening.feature_scores) if screening.feature_scores else {}
    explanation = json.loads(screening.explanation) if screening.explanation else {}
    matched = json.loads(screening.matched_skills) if screening.matched_skills else []
    missing = json.loads(screening.missing_skills) if screening.missing_skills else []
    strengths = json.loads(screening.strengths) if screening.strengths else []
    weaknesses = json.loads(screening.weaknesses) if screening.weaknesses else []

    return ScreeningResultResponse(
        id=screening.id,
        resume_id=screening.resume_id,
        job_id=screening.job_id,
        candidate_name=candidate_name,
        match_score=screening.match_score,
        prediction=screening.prediction,
        probability=screening.probability,
        feature_scores=FeatureScores(**feature_scores),
        explanation=ExplanationDetail(**explanation),
        matched_skills=matched,
        missing_skills=missing,
        strengths=strengths,
```

```
        weaknesses=weaknesses,
        created_at=screening.created_at,
    )
```

**models.py:**

```
import json
from datetime import datetime

from sqlalchemy import Column, Integer, String, Float, Text, DateTime, ForeignKey
from sqlalchemy.orm import relationship

from backend.core.database import Base

class Resume(Base):
    __tablename__ = "resumes"

    id = Column(Integer, primary_key=True, index=True)
    name = Column(String(255), nullable=True)
    email = Column(String(255), nullable=True)
    phone = Column(String(50), nullable=True)
    skills = Column(Text, default="[]")  # JSON list
    education = Column(Text, nullable=True)
    experience_years = Column(Float, default=0)
    projects = Column(Text, default="[]")  # JSON list
    certifications = Column(Text, default="[]")  # JSON list
    file_path = Column(String(500), nullable=True)
    file_name = Column(String(255), nullable=True)
    raw_text = Column(Text, nullable=True)
    created_at = Column(DateTime, default=datetime.utcnow)

    screening_results = relationship("ScreeningResult", back_populates="resume")

    @property
    def skills_list(self):
        return json.loads(self.skills) if self.skills else []

    @skills_list.setter
    def skills_list(self, value):
        self.skills = json.dumps(value)

    @property
```

```
    def projects_list(self):
        return json.loads(self.projects) if self.projects else []

    @projects_list.setter
    def projects_list(self, value):
        self.projects = json.dumps(value)

    @property
    def certifications_list(self):
        return json.loads(self.certifications) if self.certifications else []

    @certifications_list.setter
    def certifications_list(self, value):
        self.certifications = json.dumps(value)

class JobDescription(Base):
    __tablename__ = "job_descriptions"

    id = Column(Integer, primary_key=True, index=True)
    title = Column(String(255), nullable=False)
    description = Column(Text, nullable=False)
    skills_required = Column(Text, default="[]")  # JSON list
    skills_preferred = Column(Text, default="[]")  # JSON list
    experience_required = Column(Float, default=0)
    education_required = Column(String(255), nullable=True)
    created_at = Column(DateTime, default=datetime.utcnow)

    screening_results = relationship("ScreeningResult", back_populates="job")

    @property
    def skills_required_list(self):
        return json.loads(self.skills_required) if self.skills_required else []

    @skills_required_list.setter
    def skills_required_list(self, value):
        self.skills_required = json.dumps(value)

    @property
    def skills_preferred_list(self):
        return json.loads(self.skills_preferred) if self.skills_preferred else []

    @skills_preferred_list.setter
```

```
    def skills_preferred_list(self, value):
        self.skills_preferred = json.dumps(value)

class ScreeningResult(Base):
    __tablename__ = "screening_results"

    id = Column(Integer, primary_key=True, index=True)
    resume_id = Column(Integer, ForeignKey("resumes.id"), nullable=False)
    job_id = Column(Integer, ForeignKey("job_descriptions.id"), nullable=False)
    match_score = Column(Float, default=0.0)
    prediction = Column(String(50), default="Pending")
    probability = Column(Float, default=0.0)
    explanation = Column(Text, default="{}")  # JSON dict
    feature_scores = Column(Text, default="{}")  # JSON dict
    strengths = Column(Text, default="[]")  # JSON list
    weaknesses = Column(Text, default="[]")  # JSON list
    matched_skills = Column(Text, default="[]")  # JSON list
    missing_skills = Column(Text, default="[]")  # JSON list
    created_at = Column(DateTime, default=datetime.utcnow)

    resume = relationship("Resume", back_populates="screening_results")
    job = relationship("JobDescription", back_populates="screening_results")
```

**Frontend Logic (The UI)**

**App.jsx:**

```
import { BrowserRouter, Routes, Route, NavLink } from 'react-router-dom';
import LandingPage from './pages/LandingPage';
import UploadPage from './pages/UploadPage';
import ResultsPage from './pages/ResultsPage';
import ComparePage from './pages/ComparePage';
import RankingPage from './pages/RankingPage';

function AppNavbar() {
  return (
    <nav className="navbar">
      <div className="navbar-inner">
        <NavLink to="/" className="logo">
          <div className="logo-icon-wrap">
            <svg width="36" height="36" viewBox="0 0 36 36" fill="none">
              <defs>
                <linearGradient id="navLogoGrad" x1="0" y1="0" x2="36" y2="36">
```

```
                <stop offset="0%" stopColor="#6366f1" />
                <stop offset="100%" stopColor="#a855f7" />
              </linearGradient>
            </defs>
            <rect width="36" height="36" rx="10" fill="url(#navLogoGrad)" />
            <path d="M10 18h6l2.5-5 3 10 2.5-5h4" stroke="#fff" strokeWidth="2.2"
strokeLinecap="round" strokeLinejoin="round" fill="none"/>
            <circle cx="28" cy="10" r="3" fill="#22c55e" opacity="0.9"/>
          </svg>
        </div>
        <div>
          <div className="logo-text">HireLens AI</div>
        </div>
      </NavLink>
      <div className="nav-links">
        <NavLink to="/upload-resume" className={({isActive}) => `nav-link ${isActive ?
'active' : ''}`}>
          Upload
        </NavLink>
        <NavLink to="/results" className={({isActive}) => `nav-link ${isActive ? 'active' :
''}`}>
          Results
        </NavLink>
        <NavLink to="/compare" className={({isActive}) => `nav-link ${isActive ? 'active' :
''}`}>
          Compare
        </NavLink>
        <NavLink to="/ranking" className={({isActive}) => `nav-link ${isActive ? 'active' :
''}`}>
          Rankings
        </NavLink>
      </div>
    </div>
  </nav>
 );
}

function AppLayout({ children }) {
 return (
  <>
    <AppNavbar />
    <main className="main-content">{children}</main>
  </>
```

```
  );
}

function App() {
  return (
    <BrowserRouter>
      <Routes>
        {/* Landing page — full-width, no app navbar */}
        <Route path="/" element={<LandingPage />} />

        {/* App pages — wrapped with app navbar */}
        <Route path="/upload-resume" element={<AppLayout><UploadPage /></AppLayout>}
/>
        <Route path="/results" element={<AppLayout><ResultsPage /></AppLayout>} />
        <Route path="/compare" element={<AppLayout><ComparePage /></AppLayout>} />
        <Route path="/ranking" element={<AppLayout><RankingPage /></AppLayout>} />
      </Routes>
    </BrowserRouter>
  );
}

export default App;
```

**api.js:**

```
import axios from 'axios';

const api = axios.create({
  baseURL: '/api',
  timeout: 60000,
});

// ─── Resume APIs ──────────────────────────────────────────────────────────

export const uploadResume = async (file) => {
  const formData = new FormData();
  formData.append('file', file);
  const { data } = await api.post('/upload_resume', formData, {
    headers: { 'Content-Type': 'multipart/form-data' },
  });
  return data;
```

```
};

export const getResumes = async () => {
  const { data } = await api.get('/resumes');
  return data;
};

export const deleteResume = async (resumeId) => {
  const { data } = await api.delete(`/resumes/${resumeId}`);
  return data;
};

// ——— Job APIs
————————————————————————————————————————————————————————————————————————————

export const uploadJob = async (title, description) => {
  const { data } = await api.post('/upload_job', { title, description });
  return data;
};

export const getJobs = async () => {
  const { data } = await api.get('/jobs');
  return data;
};

export const deleteJob = async (jobId) => {
  const { data } = await api.delete(`/jobs/${jobId}`);
  return data;
};

// ——— Matching APIs
————————————————————————————————————————————————————————————————————————————

export const matchResume = async (resumeId, jobId) => {
  const { data } = await api.post('/match_resume', {
    resume_id: resumeId,
    job_id: jobId,
  });
  return data;
};

export const matchBatch = async (resumeIds, jobId) => {
```

```
  const { data } = await api.post('/match_batch', {
    resume_ids: resumeIds,
    job_id: jobId,
  });
  return data;
};

// ─── Results & Ranking APIs ──────────────────────────────────────────────

export const getResults = async (jobId) => {
  const { data } = await api.get(`/results/${jobId}`);
  return data;
};

export const getResultDetail = async (jobId, resumeId) => {
  const { data } = await api.get(`/results/${jobId}/detail/${resumeId}`);
  return data;
};

// ─── Comparison API ──────────────────────────────────────────────────────

export const compareResumes = async (resumeIdA, resumeIdB, jobId) => {
  const { data } = await api.post('/compare_resumes', {
    resume_id_a: resumeIdA,
    resume_id_b: resumeIdB,
    job_id: jobId,
  });
  return data;
};

export default api;
```

**UploadPage.jsx:**

```
import { useState, useCallback } from 'react';
import { useDropzone } from 'react-dropzone';
import { uploadResume, uploadJob, getResumes, getJobs, matchResume, matchBatch } from
'../services/api';
import ScoreGauge from '../components/ScoreGauge';
import FeatureBars from '../components/FeatureBars';
import ExplanationPanel from '../components/ExplanationPanel';
```

```
import SkillTags from '../components/SkillTags';

export default function UploadPage() {
  const [file, setFile] = useState(null);
  const [jobTitle, setJobTitle] = useState('');
  const [jobDesc, setJobDesc] = useState('');
  const [loading, setLoading] = useState(false);
  const [uploadedResume, setUploadedResume] = useState(null);
  const [uploadedJob, setUploadedJob] = useState(null);
  const [result, setResult] = useState(null);
  const [error, setError] = useState('');
  const [step, setStep] = useState(1); // 1=upload, 2=job, 3=result

  const onDrop = useCallback((acceptedFiles) => {
    if (acceptedFiles.length > 0) {
      setFile(acceptedFiles[0]);
      setError('');
    }
  }, []);

  const { getRootProps, getInputProps, isDragActive } = useDropzone({
    onDrop,
    accept: {
      'application/pdf': ['.pdf'],
      'application/vnd.openxmlformats-officedocument.wordprocessingml.document': ['.docx'],
      'text/plain': ['.txt'],
    },
    maxFiles: 1,
    maxSize: 10 * 1024 * 1024,
  });

  const handleUploadResume = async () => {
    if (!file) return;
    setLoading(true);
    setError('');
    try {
      const resume = await uploadResume(file);
      setUploadedResume(resume);
      setStep(2);
    } catch (err) {
      setError(err.response?.data?.detail || 'Failed to upload resume');
    }
```

```
  setLoading(false);
};

const handleUploadJob = async () => {
  if (!jobTitle.trim() || !jobDesc.trim()) return;
  setLoading(true);
  setError('');
  try {
    const job = await uploadJob(jobTitle, jobDesc);
    setUploadedJob(job);
    // Auto-match
    const matchResult = await matchResume(uploadedResume.id, job.id);
    setResult(matchResult);
    setStep(3);
  } catch (err) {
    setError(err.response?.data?.detail || 'Failed to process');
  }
  setLoading(false);
};

const handleStartOver = () => {
  setFile(null);
  setJobTitle('');
  setJobDesc('');
  setUploadedResume(null);
  setUploadedJob(null);
  setResult(null);
  setStep(1);
  setError('');
};

return (
  <div className="fade-in">
    {/* Step Indicator */}
    <div style={{ display: 'flex', alignItems: 'center', gap: '1rem', marginBottom: '2rem' }}>
      {[
        { num: 1, label: 'Upload Resume' },
        { num: 2, label: 'Job Description' },
        { num: 3, label: 'Results' },
      ].map((s, i) => (
        <div key={s.num} style={{ display: 'flex', alignItems: 'center', gap: '0.5rem' }}>
          <div style={{
```

```
        width: 32, height: 32, borderRadius: '50%', display: 'flex',
        alignItems: 'center', justifyContent: 'center', fontSize: '0.85rem',
        fontWeight: 700,
        background: step >= s.num ? 'var(--gradient-primary)' : 'var(--bg-card)',
        color: step >= s.num ? 'white' : 'var(--text-muted)',
        border: step >= s.num ? 'none' : '1px solid var(--border)',
      }}>
        {step > s.num ? '✓' : s.num}
      </div>
      <span style={{
        fontSize: '0.85rem', fontWeight: 600,
        color: step >= s.num ? 'var(--text-primary)' : 'var(--text-muted)',
      }}>{s.label}</span>
      {i < 2 && <div style={{
        width: 40, height: 2,
        background: step > s.num ? 'var(--accent-primary)' : 'var(--border)',
        margin: '0 0.5rem',
      }} />}
    </div>
  ))}
</div>

{error && (
  <div style={{
    padding: '12px 16px', borderRadius: 'var(--radius-sm)',
    background: 'var(--danger-bg)', border: '1px solid rgba(239,68,68,0.2)',
    color: 'var(--danger)', marginBottom: '1.5rem', fontSize: '0.9rem',
  }}>
    ⚠ {error}
  </div>
)}

{/* Step 1: Upload Resume */}
{step === 1 && (
  <div className="card scale-in">
    <div className="card-header">
      <h2 className="card-title">📄 Upload Resume</h2>
      <span style={{ color: 'var(--text-muted)', fontSize: '0.8rem' }}>
        Supported: PDF, DOCX, TXT
      </span>
    </div>
```

```
        <div {...getRootProps()} className={`upload-zone ${isDragActive ? 'active' : ''}`}>
          <input {...getInputProps()} id="resume-upload-input" />
          <div className="upload-icon">📂</div>
          <div className="upload-title">
            {isDragActive ? 'Drop your resume here...' : 'Drag & drop your resume'}
          </div>
          <div className="upload-subtitle">
            or <span className="upload-highlight">click to browse</span> files
          </div>
        </div>

        {file && (
          <div className="file-preview">
            <div className="file-icon">📄</div>
            <div className="file-info">
              <div className="file-name">{file.name}</div>
              <div className="file-size">{(file.size / 1024).toFixed(1)} KB</div>
            </div>
            <button className="btn btn-sm btn-danger" onClick={(e) => { e.stopPropagation();
setFile(null); }}>
              ✕
            </button>
          </div>
        )}

        <div style={{ marginTop: '1.5rem', display: 'flex', justifyContent: 'flex-end' }}>
          <button
            id="upload-resume-btn"
            className="btn btn-primary btn-lg"
            onClick={handleUploadResume}
            disabled={!file || loading}
          >
            {loading ? (
              <><span className="spinner" style={{ width: 18, height: 18, borderWidth: 2 }} />
Processing...</>
            ) : (
              <>Upload & Parse →</>
            )}
          </button>
        </div>
      </div>
```

```
    )}

    {/* Step 2: Job Description */}
    {step === 2 && (
      <div className="scale-in">
        {/* Resume Summary */}
        {uploadedResume && (
          <div className="card" style={{ marginBottom: '1.5rem' }}>
            <div className="card-header">
              <h3 className="card-title">✅ Resume Parsed: {uploadedResume.name}</h3>
              <span className="card-badge badge-success">Parsed</span>
            </div>
            <div style={{ display: 'flex', gap: '2rem', flexWrap: 'wrap', fontSize: '0.9rem' }}>
              {uploadedResume.email && <div><span style={{ color: 'var(--text-muted)'
}}>Email:</span> {uploadedResume.email}</div>}
              {uploadedResume.experience_years > 0 && <div><span style={{ color: 'var(--text-
muted)' }}>Experience:</span> {uploadedResume.experience_years} years</div>}
              {uploadedResume.education && <div><span style={{ color: 'var(--text-muted)'
}}>Education:</span> {uploadedResume.education}</div>}
            </div>
            {uploadedResume.skills?.length > 0 && (
              <div style={{ marginTop: '1rem' }}>
                <div style={{ fontSize: '0.8rem', color: 'var(--text-muted)', marginBottom: '0.5rem'
}}>DETECTED SKILLS</div>
                <SkillTags skills={uploadedResume.skills} type="matched" />
              </div>
            )}
          </div>
        )}

        {/* Job Description Form */}
        <div className="card">
          <div className="card-header">
            <h2 className="card-title">💼 Enter Job Description</h2>
          </div>
          <div className="form-group">
            <label className="form-label" htmlFor="job-title-input">Job Title</label>
            <input
              id="job-title-input"
              className="form-input"
              type="text"
              placeholder="e.g. Senior Python Developer"
```

```
            value={jobTitle}
            onChange={(e) => setJobTitle(e.target.value)}
          />
        </div>
        <div className="form-group">
          <label className="form-label" htmlFor="job-desc-input">Job Description</label>
          <textarea
            id="job-desc-input"
            className="form-textarea"
            placeholder="Paste the full job description here..."
            value={jobDesc}
            onChange={(e) => setJobDesc(e.target.value)}
            rows={8}
          />
        </div>
        <div style={{ display: 'flex', justifyContent: 'space-between' }}>
          <button className="btn btn-secondary" onClick={() => setStep(1)}>
            ← Back
          </button>
          <button
            id="analyze-match-btn"
            className="btn btn-primary btn-lg"
            onClick={handleUploadJob}
            disabled={!jobTitle.trim() || !jobDesc.trim() || loading}
          >
            {loading ? (
              <><span className="spinner" style={{ width: 18, height: 18, borderWidth: 2 }}
/> Analyzing...</>
            ) : (
              <>Analyze Match →</>
            )}
          </button>
        </div>
      </div>
    </div>
  )}

  {/* Step 3: Results */}
  {step === 3 && result && (
    <div className="scale-in">
      <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center',
marginBottom: '1.5rem' }}>
```

```
          <div>
            <h2 className="page-title">Screening Results</h2>
            <p className="page-subtitle">
              {result.candidate_name} → {uploadedJob?.title}
            </p>
          </div>
          <button className="btn btn-primary" onClick={handleStartOver}>
            + New Screening
          </button>
        </div>

        {/* Score + Prediction Row */}
        <div className="grid-3" style={{ marginBottom: '1.5rem' }}>
          <div className="card" style={{ display: 'flex', justifyContent: 'center' }}>
            <ScoreGauge score={result.match_score} label="Match Score" />
          </div>
          <div className="card" style={{ display: 'flex', flexDirection: 'column', alignItems:
'center', justifyContent: 'center' }}>
            <div style={{
              fontSize: '1.5rem', fontWeight: 800,
              color: result.prediction === 'Selected' ? 'var(--success)' : 'var(--danger)',
              marginBottom: '0.5rem',
            }}>
              {result.prediction === 'Selected' ? '✅' : '❌'} {result.prediction}
            </div>
            <div style={{ color: 'var(--text-muted)', fontSize: '0.85rem' }}>
              Confidence: {(result.probability * 100).toFixed(0)}%
            </div>
          </div>
          <div className="card">
            <h3 className="card-title" style={{ marginBottom: '1rem' }}>Feature Scores</h3>
            <FeatureBars scores={result.feature_scores} />
          </div>
        </div>

        {/* Skills + Explanation Row */}
        <div className="grid-2" style={{ marginBottom: '1.5rem' }}>
          <div className="card">
            <h3 className="card-title" style={{ marginBottom: '1rem' }}>🎯 Skills
Analysis</h3>
            {result.matched_skills?.length > 0 && (
              <div style={{ marginBottom: '1rem' }}>
```

```
              <div style={{ fontSize: '0.8rem', color: 'var(--text-muted)', marginBottom: '0.5rem'
}}>MATCHED SKILLS</div>
              <SkillTags skills={result.matched_skills} type="matched" />
            </div>
          )}
          {result.missing_skills?.length > 0 && (
            <div>
              <div style={{ fontSize: '0.8rem', color: 'var(--text-muted)', marginBottom: '0.5rem'
}}>MISSING SKILLS</div>
              <SkillTags skills={result.missing_skills} type="missing" />
            </div>
          )}
        </div>
        <div className="card">
          <h3 className="card-title" style={{ marginBottom: '1rem' }}>💡 Explanation</h3>
          <ExplanationPanel explanation={result.explanation} />
        </div>
      </div>

      {/* Strengths & Weaknesses */}
      <div className="grid-2">
        <div className="card">
          <h3 className="card-title" style={{ marginBottom: '1rem', color: 'var(--success)'
}}>💪 Strengths</h3>
          {result.strengths?.map((s, i) => (
            <div key={i} className="explanation-item positive">
              <span className="explanation-icon">✓</span>
              <span>{s}</span>
            </div>
          ))}
          {(!result.strengths || result.strengths.length === 0) && (
            <div style={{ color: 'var(--text-muted)', fontSize: '0.9rem' }}>No specific strengths
identified</div>
          )}
        </div>
        <div className="card">
          <h3 className="card-title" style={{ marginBottom: '1rem', color: 'var(--danger)'
}}>⚠ Weaknesses</h3>
          {result.weaknesses?.map((w, i) => (
            <div key={i} className="explanation-item negative">
              <span className="explanation-icon">✗</span>
              <span>{w}</span>
```

```
            </div>
          ))}
          {(!result.weaknesses || result.weaknesses.length === 0) && (
            <div style={{ color: 'var(--text-muted)', fontSize: '0.9rem' }}>No significant
weaknesses found</div>
          )}
        </div>
      </div>
    </div>
  )}
 </div>
);
}
```

**ResultsPage.jsx:**

```
import { useState, useEffect } from 'react';
import { getJobs, getResumes, getResults, getResultDetail } from '../services/api';
import ScoreGauge from '../components/ScoreGauge';
import FeatureBars from '../components/FeatureBars';
import ExplanationPanel from '../components/ExplanationPanel';
import SkillTags from '../components/SkillTags';

export default function ResultsPage() {
  const [jobs, setJobs] = useState([]);
  const [selectedJob, setSelectedJob] = useState(null);
  const [rankings, setRankings] = useState(null);
  const [selectedResult, setSelectedResult] = useState(null);
  const [loading, setLoading] = useState(false);

  useEffect(() => {
    loadJobs();
  }, []);

  const loadJobs = async () => {
    try {
      const data = await getJobs();
      setJobs(data.jobs || []);
    } catch (err) {
      console.error('Failed to load jobs', err);
    }
```

```
  };

  const handleSelectJob = async (jobId) => {
    setLoading(true);
    setSelectedJob(jobId);
    setSelectedResult(null);
    try {
      const data = await getResults(jobId);
      setRankings(data);
    } catch (err) {
      console.error('Failed to load results', err);
    }
    setLoading(false);
  };

  const handleSelectCandidate = async (resumeId) => {
    setLoading(true);
    try {
      const data = await getResultDetail(selectedJob, resumeId);
      setSelectedResult(data);
    } catch (err) {
      console.error('Failed to load result detail', err);
    }
    setLoading(false);
  };

  return (
    <div className="fade-in">
      <div className="page-header">
        <h1 className="page-title">📊 Screening Results</h1>
        <p className="page-subtitle">View detailed analysis for each resume-job match</p>
      </div>

      {/* Job Selector */}
      <div className="card" style={{ marginBottom: '1.5rem' }}>
        <div className="form-group" style={{ marginBottom: 0 }}>
          <label className="form-label" htmlFor="results-job-select">Select Job
Description</label>
          <select
            id="results-job-select"
            className="form-select"
            value={selectedJob || ''}
```

```
          onChange={(e) => handleSelectJob(Number(e.target.value))}
        >
          <option value="">-- Choose a job --</option>
          {jobs.map((j) => (
            <option key={j.id} value={j.id}>{j.title}</option>
          ))}
        </select>
      </div>
    </div>

    {loading && (
      <div className="loading-container">
        <div className="spinner" />
        <div className="loading-text">Loading results...</div>
      </div>
    )}

    {!loading && !selectedJob && (
      <div className="empty-state">
        <div className="empty-icon">📋</div>
        <div className="empty-title">Select a Job Description</div>
        <div className="empty-subtitle">Choose a job from the dropdown above to view
screening results</div>
      </div>
    )}

    {!loading && selectedJob && rankings && (
      <div className="grid-2">
        {/* Left: Candidate List */}
        <div>
          <h3 style={{ marginBottom: '1rem', fontSize: '1rem', fontWeight: 600 }}>
            Candidates ({rankings.rankings?.length || 0})
          </h3>
          {rankings.rankings?.length > 0 ? (
            <div style={{ display: 'flex', flexDirection: 'column', gap: '0.75rem' }}>
              {rankings.rankings.map((r) => (
                <div
                  key={r.resume_id}
                  className="ranking-card"
                  onClick={() => handleSelectCandidate(r.resume_id)}
                  style={{
                    borderColor: selectedResult?.resume_id === r.resume_id ? 'var(--accent-
```

```
primary)' : undefined,
                }}
              >
                <div className={`ranking-position rank-${r.rank <= 3 ? r.rank : 'other'}`}>
                  #{r.rank}
                </div>
                <div className="ranking-info">
                  <div className="ranking-name">{r.candidate_name}</div>
                  <div className="ranking-prediction">
                    <span className={`card-badge ${r.prediction === 'Selected' ? 'badge-success'
: 'badge-danger'}`}>
                      {r.prediction}
                    </span>
                  </div>
                </div>
                <div className="ranking-score" style={{
                  color: r.match_score >= 70 ? 'var(--success)' : r.match_score >= 50 ? 'var(--
warning)' : 'var(--danger)',
                }}>
                  {r.match_score}%
                </div>
              </div>
            ))}
          </div>
        ) : (
          <div className="empty-state" style={{ padding: '2rem' }}>
            <div className="empty-icon">🔍</div>
            <div className="empty-title">No Results Yet</div>
            <div className="empty-subtitle">Upload resumes and run matching from the
Upload page</div>
          </div>
        )}
      </div>

      {/* Right: Detailed View */}
      <div>
        {selectedResult ? (
          <div className="slide-in">
            <div className="card" style={{ marginBottom: '1rem' }}>
              <div style={{ display: 'flex', alignItems: 'center', gap: '1.5rem' }}>
                <ScoreGauge score={selectedResult.match_score} size={100} label="Score" />
                <div>
```

```
                <h3 style={{ fontSize: '1.2rem', fontWeight: 700
}}>{selectedResult.candidate_name}</h3>
                <div style={{
                  fontSize: '1rem', fontWeight: 700, marginTop: '0.25rem',
                  color: selectedResult.prediction === 'Selected' ? 'var(--success)' : 'var(--
danger)',
                }}>
                  {selectedResult.prediction === 'Selected' ? '✅' : '❌'}
{selectedResult.prediction}
                </div>
              </div>
            </div>
          </div>

          <div className="card" style={{ marginBottom: '1rem' }}>
            <h4 style={{ marginBottom: '0.75rem', fontWeight: 600, fontSize: '0.95rem'
}}>Feature Scores</h4>
            <FeatureBars scores={selectedResult.feature_scores} />
          </div>

          <div className="card" style={{ marginBottom: '1rem' }}>
            <h4 style={{ marginBottom: '0.75rem', fontWeight: 600, fontSize: '0.95rem'
}}>Skills</h4>
            {selectedResult.matched_skills?.length > 0 && (
              <div style={{ marginBottom: '0.75rem' }}>
                <div style={{ fontSize: '0.75rem', color: 'var(--text-muted)', marginBottom:
'0.4rem' }}>MATCHED</div>
                <SkillTags skills={selectedResult.matched_skills} type="matched" />
              </div>
            )}
            {selectedResult.missing_skills?.length > 0 && (
              <div>
                <div style={{ fontSize: '0.75rem', color: 'var(--text-muted)', marginBottom:
'0.4rem' }}>MISSING</div>
                <SkillTags skills={selectedResult.missing_skills} type="missing" />
              </div>
            )}
          </div>

          <div className="card">
            <h4 style={{ marginBottom: '0.75rem', fontWeight: 600, fontSize: '0.95rem'
}}>Explanation</h4>
            <ExplanationPanel explanation={selectedResult.explanation} />
```

```
            </div>
          </div>
        ) : (
          <div className="empty-state" style={{ padding: '3rem' }}>
            <div className="empty-icon">👈</div>
            <div className="empty-title">Select a Candidate</div>
            <div className="empty-subtitle">Click on a candidate to see detailed
results</div>
          </div>
        )}
      </div>
    </div>
  )}
 </div>
);
}
```

## 5.3 Architectural Components

**1. AI Package (ai/)** — The intelligence layer of HireLens AI, comprising six independent Python packages:

- ai/resume_parser/parser.py — Multi-format resume parsing engine. Responsible for all text extraction and structured field identification from uploaded candidate files. This module is the entry point of the AI pipeline and feeds all downstream components.
- ai/job_parser/parser.py — Job description parsing engine. Transforms free-form recruiter text into a structured profile of technical requirements, converting unstructured intent into machine-computable fields.
- ai/feature_extractor/extractor.py — Feature engineering module. Converts the structured resume and job data into five normalised numerical scores, each representing a specific dimension of candidate–job compatibility. This module is the computational heart of the AI pipeline.
- ai/matching_engine/matching.py — Score aggregation and prediction engine. Applies the configurable weighted formula to produce a final match score, applies the selection threshold, and orchestrates the generation of strengths, weaknesses, and the explainability report.

- ai/explainability/explainer.py — Transparency engine. Decomposes the final score into feature-level contributions, generates natural language descriptions of each contribution, and constructs the complete human-readable explanation delivered to the recruiter.
- ai/comparison/comparator.py — Comparative analysis engine. Runs the full matching pipeline for two candidates independently and synthesises the results into a structured side-by-side comparison with pros, cons, and a natural language hiring recommendation.

**2. Backend Package (backend/)** — The web API layer:

- backend/main.py — FastAPI application factory. Configures CORS middleware, mounts static file serving for uploaded resumes, registers the API router, and initialises the database schema on startup.
- backend/api/routes.py — REST API route definitions. Implements 12 endpoints covering resume upload/list/delete, job description create/list/delete, single and batch matching, result retrieval, ranked result listing, and candidate comparison.
- backend/core/config.py — Centralised configuration. Defines all system parameters including file storage paths, allowed extensions, file size limits, CORS origins, matching weights, and the selection threshold.
- backend/core/database.py — Database session management. Configures the SQLAlchemy engine, session factory, and declarative base. Provides the get_db() dependency injector used by all route handlers.
- backend/models/models.py — ORM model definitions. Declares the Resume, JobDescription, and ScreeningResult SQLAlchemy models with all columns and relationship declarations. Includes Python property accessors that transparently serialise and deserialise JSON list columns.
- backend/schemas/schemas.py — Pydantic schema definitions. Defines request validation schemas (JobDescriptionCreate, MatchRequest, MatchBatchRequest, CompareRequest) and response serialisation schemas for all API endpoints, ensuring type safety at the API boundary.

**3. Frontend Package (frontend/src/)** — The presentation layer:

- App.jsx — Root application component. Configures the React Router DOM with five routes and wraps all non-landing pages in a shared AppLayout that includes the application navigation bar.
- pages/LandingPage.jsx — Marketing landing page assembling the Navbar, Hero, Features, HowItWorks, ExplainableSection, CTASection, Footer, and ScrollToTop components.
- pages/UploadPage.jsx — Primary user workflow page. Implements the three-step guided workflow (resume upload → job description → results) using react-dropzone for file selection and the ScoreGauge, FeatureBars, ExplanationPanel, and SkillTags components to present results.
- pages/ResultsPage.jsx — Historical results viewer. Allows job selection and candidate selection to view detailed screening results stored in the database from previous sessions.
- pages/ComparePage.jsx — Candidate comparison page. Accepts job, Candidate A, and Candidate B selections and renders a side-by-side CandidateColumn layout with scores, metrics, pros, cons, skills, and the AI recommendation.
- pages/RankingPage.jsx — Ranked leaderboard page. Supports batch matching of all uploaded resumes against a selected job and displays all candidates in descending score order with aggregate statistics (total, selected, rejected, average score).
- services/api.js — HTTP client module. Exports typed async functions for every backend API endpoint using Axios, serving as the single communication boundary between the React frontend and the FastAPI backend.

## 5.4 Feature Extraction

Feature extraction in HireLens AI is the process of converting unstructured resume and job description text into a set of five quantitative, normalised signals that represent the degree of candidate fit across distinct evaluation dimensions. Each feature is independently computed and contributes a specific, bounded value to the final match score.

**Skill Coverage Feature:** Skill extraction begins by normalising both the candidate's skill list and the job's combined required and preferred skill list to lowercase. A set intersection operation identifies matched skills, while set difference identifies missing ones. The skill score is the ratio |matched| / |all_job_skills|, ranging continuously from 0.0 (no overlap) to 1.0 (perfect coverage). This feature carries the highest weight (40%) in the final score, reflecting

the industry consensus that technical skill alignment is the primary determinant of initial candidate qualification.

**Experience Relevance Feature:** Experience years extracted from the resume are compared to the minimum requirement extracted from the job description. The score is computed as resume_exp / required_exp, capped at 1.0 to prevent over-scoring. For jobs without explicit experience requirements, a scaled absolute score is computed. This feature is weighted at 25%, acknowledging that experience is a critical but secondary qualifier after skills.

**Education Match Feature:** Education levels are mapped to an ordinal hierarchy (1–4), and candidate level is compared to required level. An exact match or over-qualification yields 1.0; one level below yields 0.6; further below yields 0.3. When no requirement is specified, a default score of 0.7 (any education) or 0.5 (no education found) is assigned. This dimension is weighted at 15%, the lowest primary weight, reflecting that many technical roles accept equivalent experience in lieu of formal education.

**Project Relevance Feature:** Project descriptions are concatenated and searched for job skill keywords. The score measures the density of job-relevant technology terms in the candidate's project portfolio, normalised by 30% of the total job skill count to avoid penalising candidates with deep but narrowly relevant project experience. This feature (20% weight) captures practical, hands-on application of skills beyond their mere listing.

**Certification Score Feature:** Certifications are treated as a supplementary signal rather than a primary feature. Relevant certifications (those containing job skill terms) contribute a bonus of up to 5% on top of the weighted primary score, rather than being a weighted primary dimension. This design choice ensures that certifications enhance a strong candidate's score without introducing an artificial barrier for candidates without certifications.

## 5.5 Packages / Libraries Used

### Backend Libraries

- **FastAPI (0.115.0):** High-performance Python framework for building REST APIs with automatic documentation.
- **Uvicorn (0.30.0):** ASGI server used to run the FastAPI application with async support.

- **SQLAlchemy (2.0.35):** ORM for database models, sessions, and queries.
- **pdfplumber (0.11.4):** Extracts text from PDF resumes with multi-page support.
- **python-docx (1.1.2):** Extracts text from DOCX files.
- **python-multipart (0.0.9):** Handles file uploads in FastAPI.
- **spaCy (3.7.6):** NLP processing including tokenization and NER.
- **sentence-transformers (3.1.0):** Generates embeddings for semantic similarity.
- **scikit-learn (1.5.2):** ML utilities like cosine similarity.
- **SHAP (0.46.0):** Feature-level explainability for model predictions.
- **LIME (0.2.0.1):** Local explanations for individual results.
- **KeyBERT (0.8.5):** Keyword extraction using BERT.
- **NumPy (1.26.4):** Numerical computations for ML workflows.
- **Pydantic (with FastAPI):** Data validation and schema management.

## Frontend Libraries

- **React (19.2.4):** Component-based UI development.
- **React DOM (19.2.4):** Renders React components in the browser.
- **React Router DOM (7.13.1):** Client-side routing without reload.
- **Vite (8.0.0):** Fast build tool and development server.
- **TailwindCSS (4.2.1):** Utility-first CSS framework for styling.
- **Axios (1.13.6):** API communication with backend services.
- **Chart.js (4.5.1):** Interactive charts and visualizations.
- **react-chartjs-2 (5.3.1):** React wrapper for Chart.js.
- **react-dropzone (15.0.0):** Drag-and-drop file upload component.

## 5.6 Output Screens

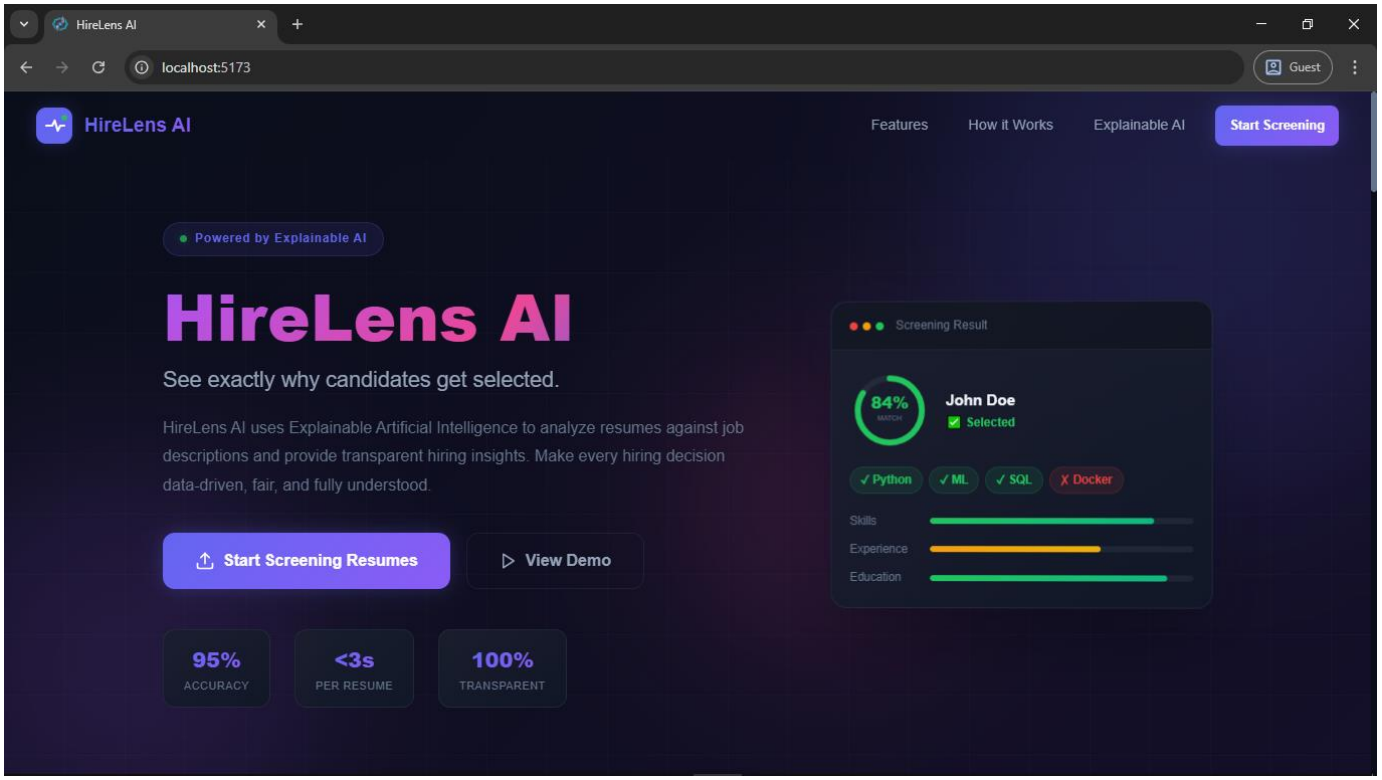


**Fig 5.1: HireLens AI Landing Page**

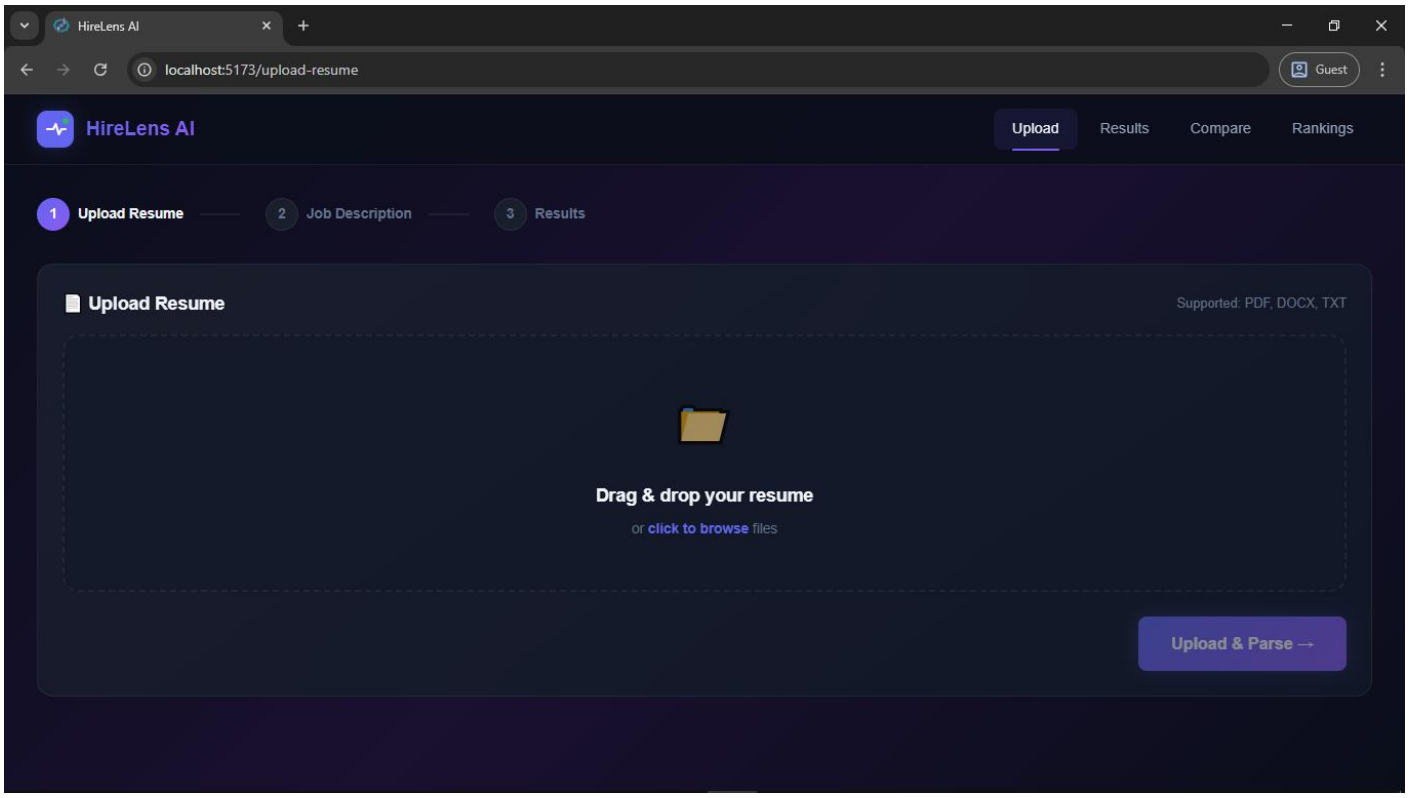


**Fig 5.2: Upload Resume Page**

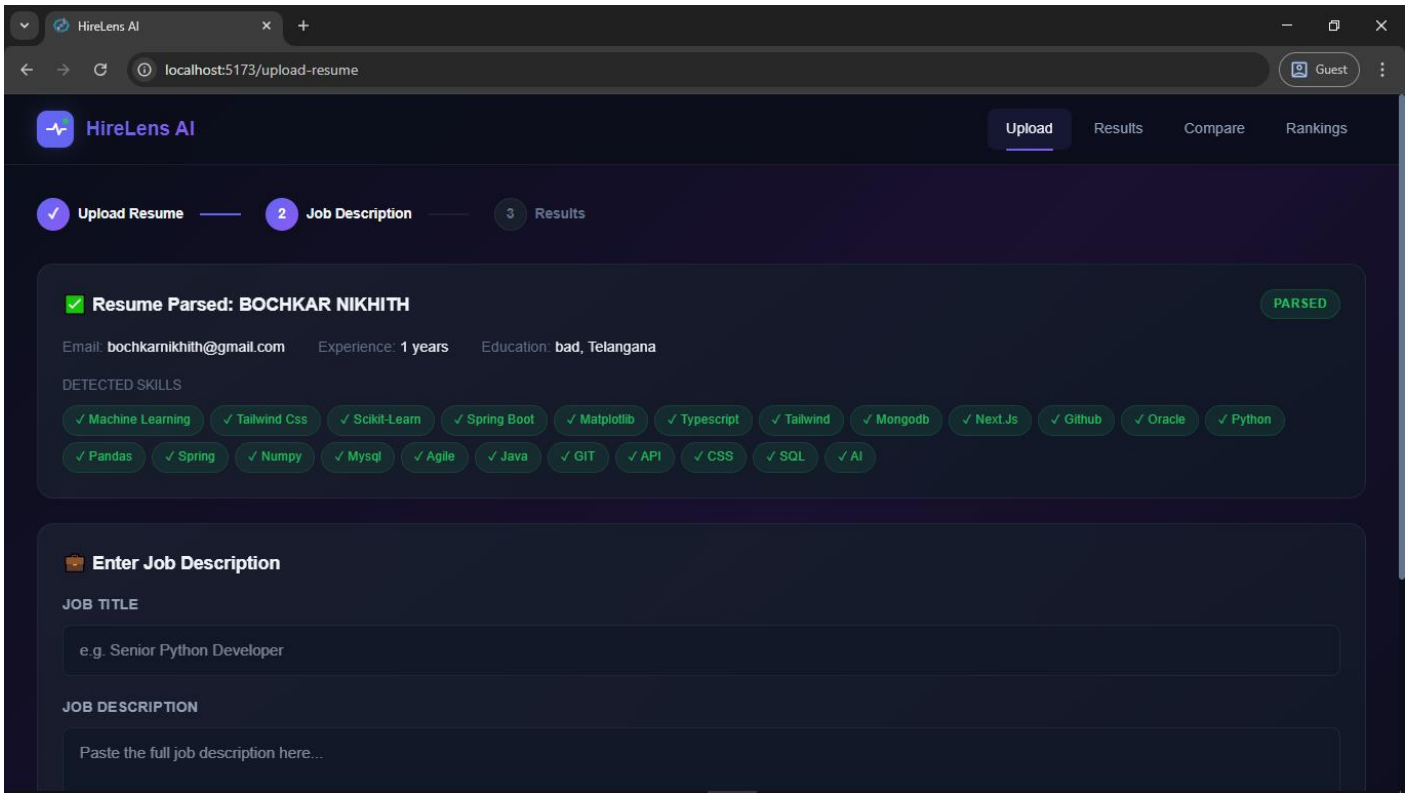


**Fig 5.3: Job Title and Job Description Page – 1**

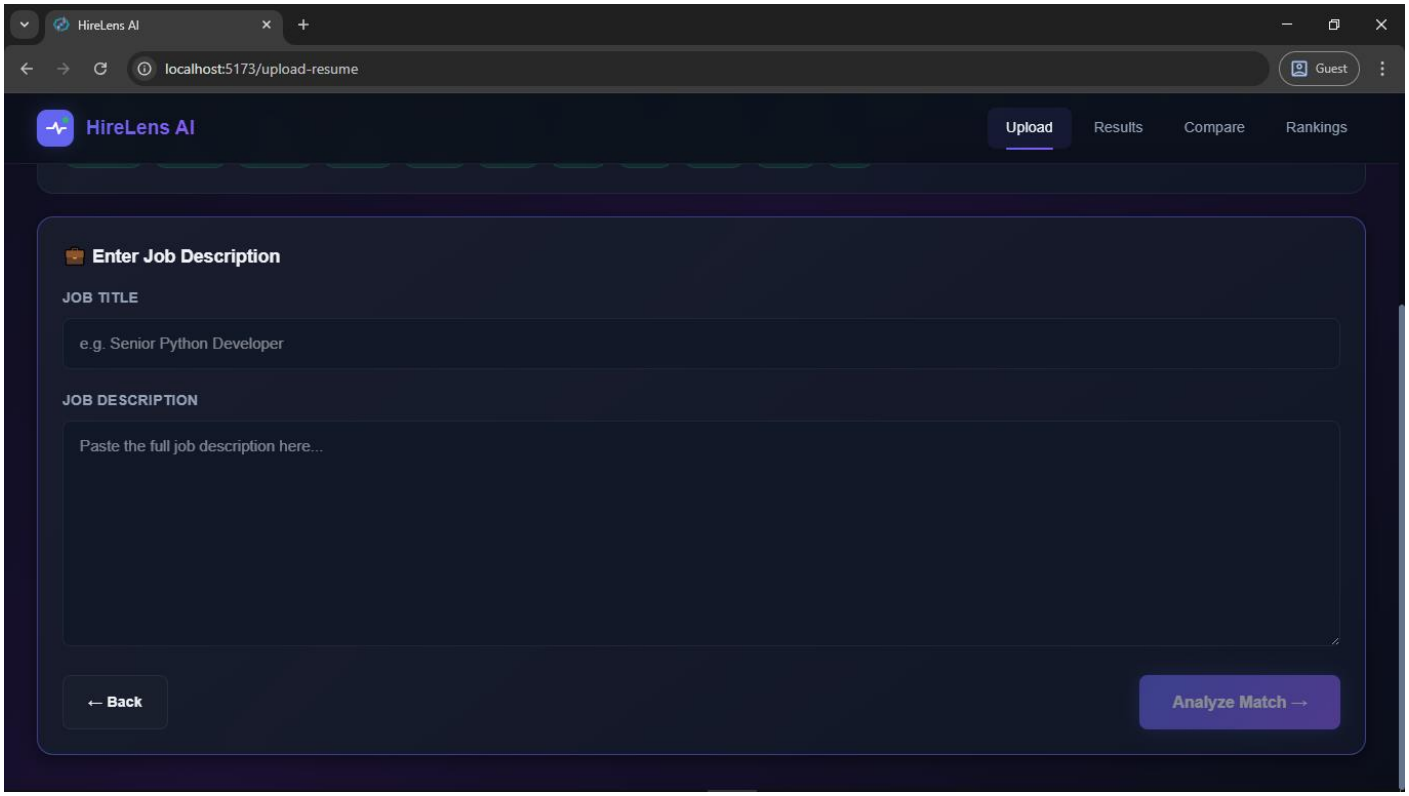


**Fig 5.4: Job Title and Job Description Page - 2**

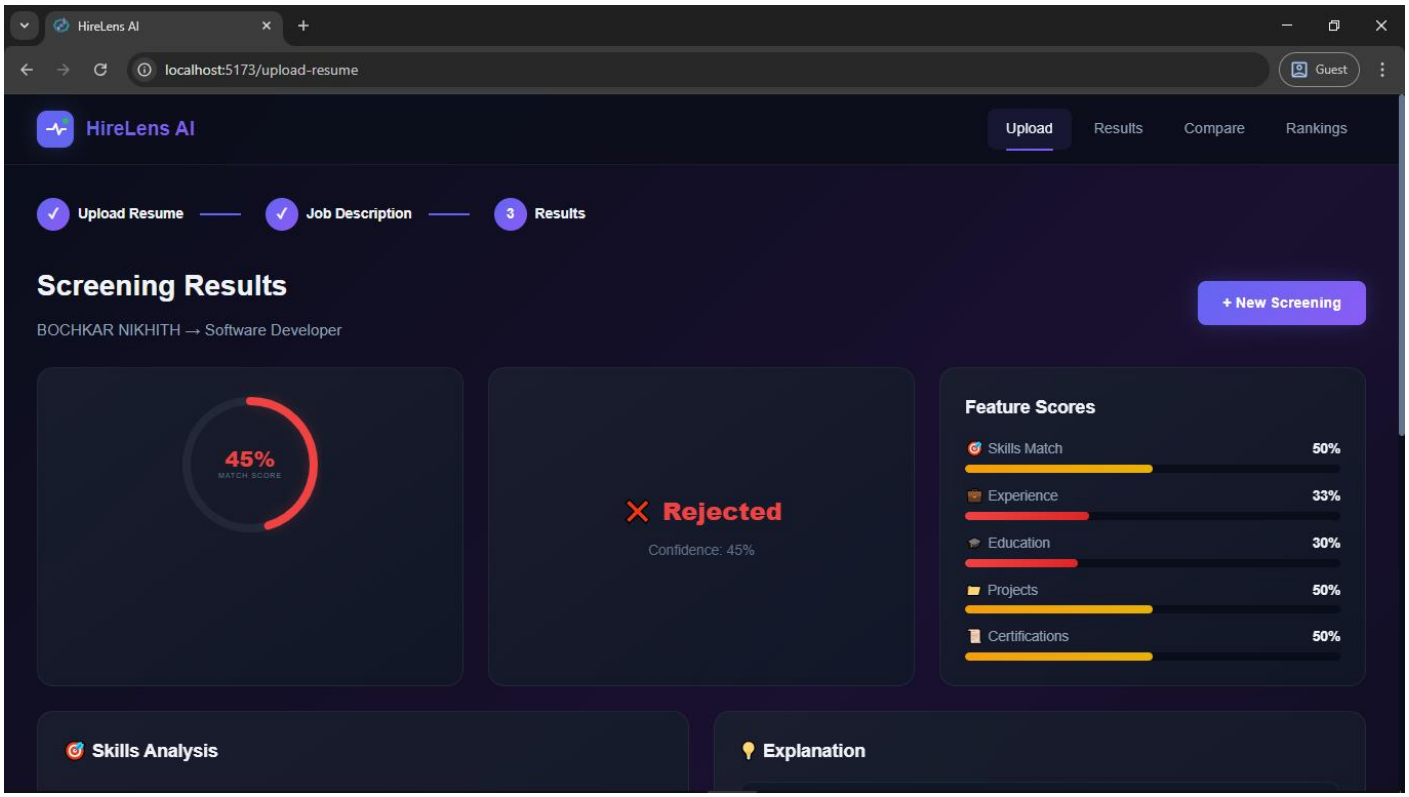


**Fig 5.5: Screening Results Page - 1**

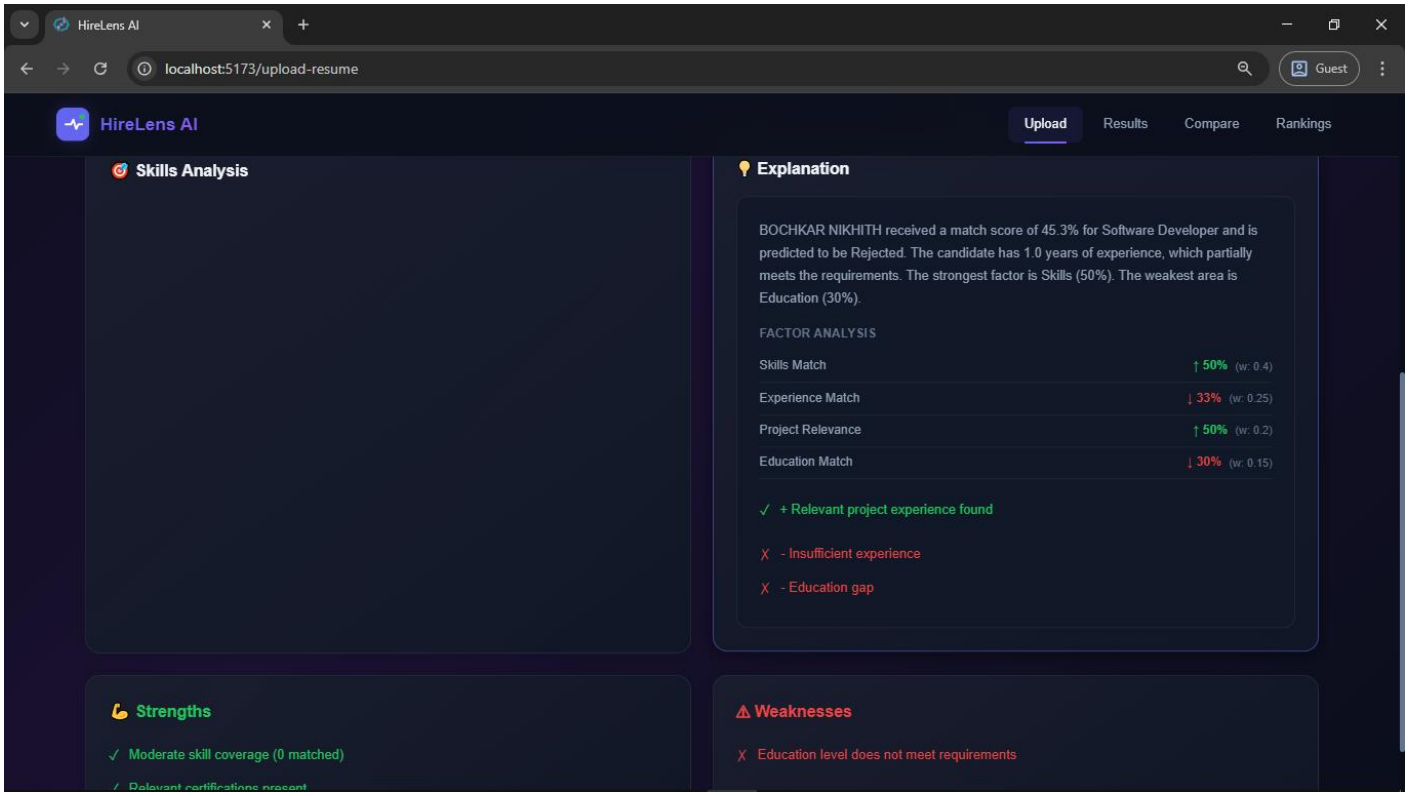


**Fig 5.6: Screening Results Page - 2**

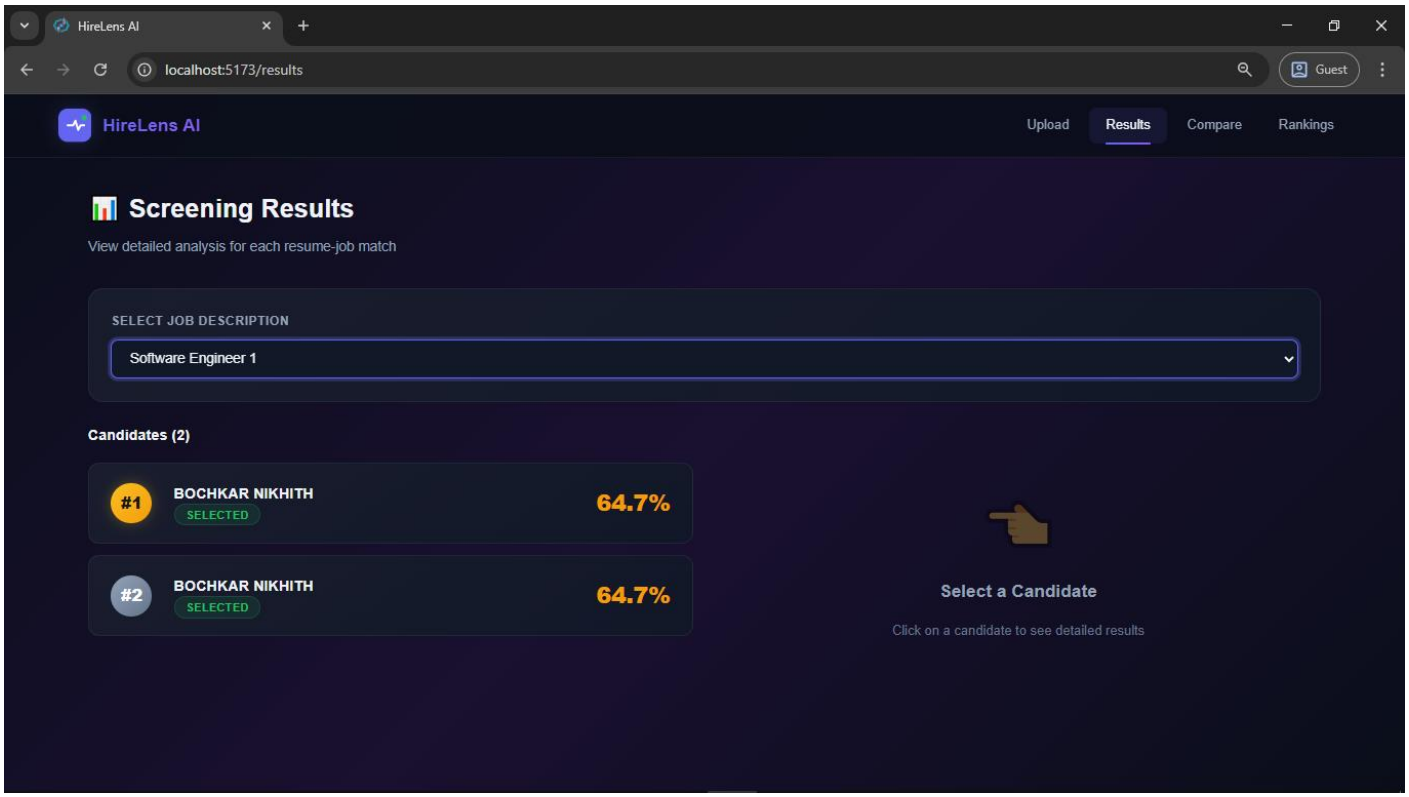


**Fig 5.7: Results Page**

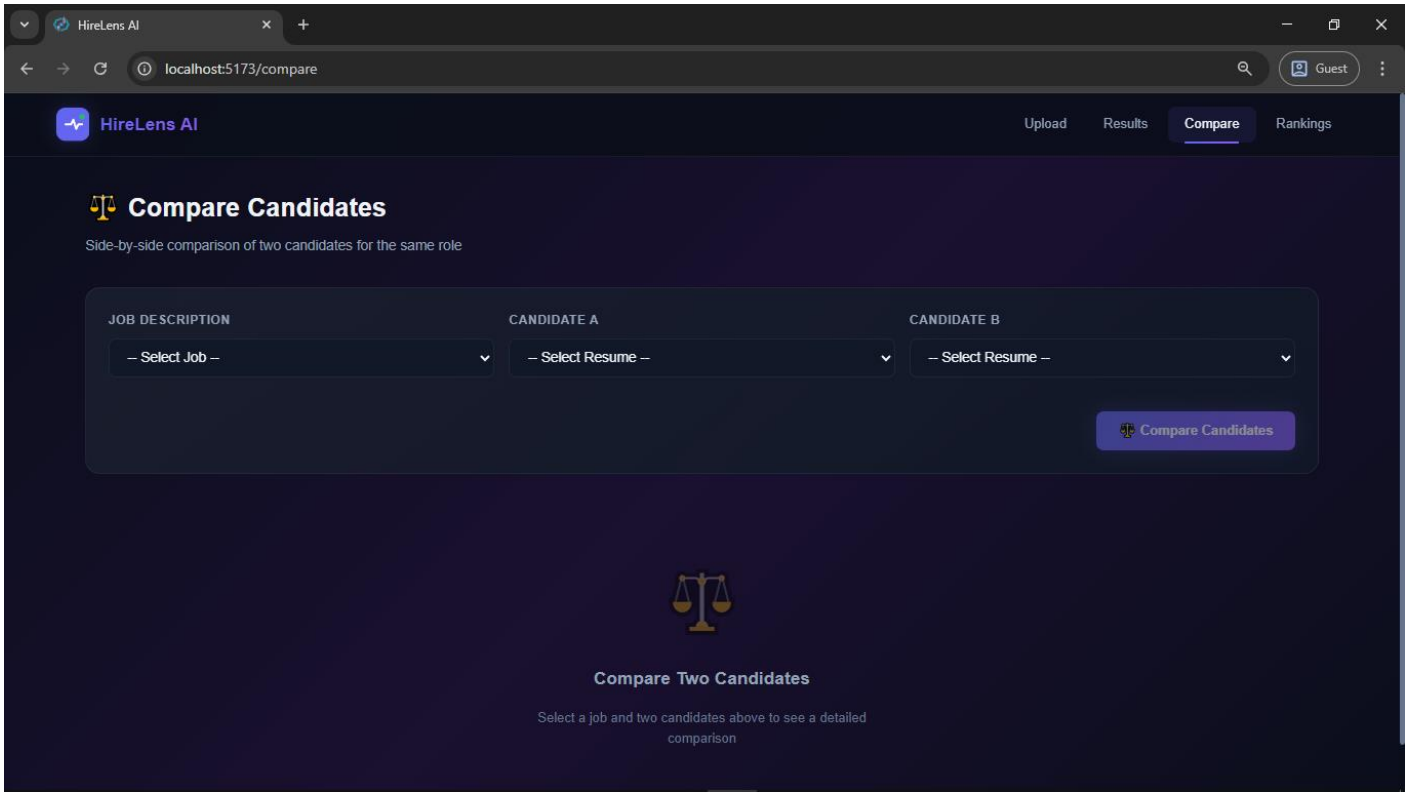


**Fig 5.8: Resume Comparison Page**

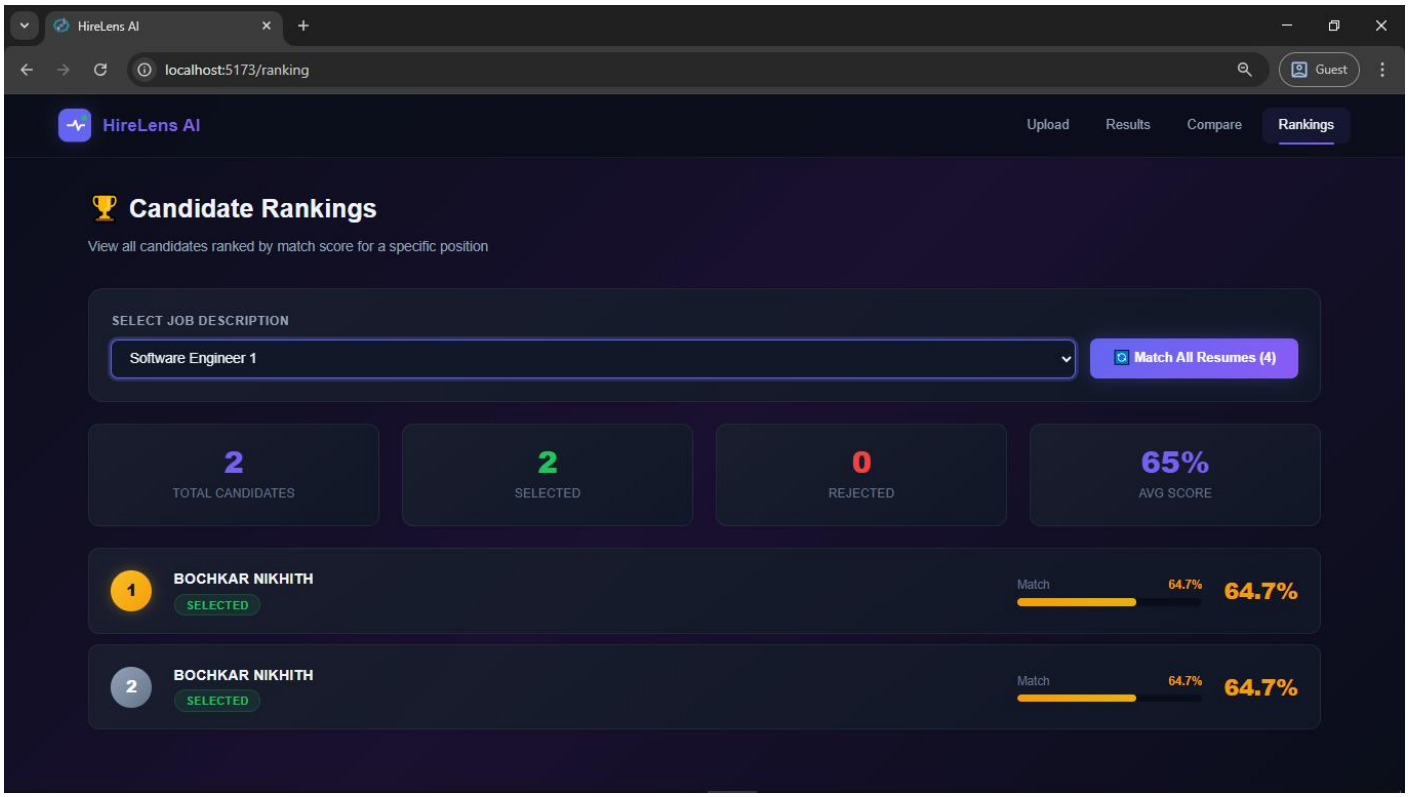


**Fig 5.9: Rankings Page**

# CHAPTER 6

# SYSTEM TESTING

System testing for HireLens AI was performed to verify that each component of the system behaves correctly in isolation and in integration, and that the end-to-end workflow produces accurate, consistent, and explainable results. The testing strategy covered functional correctness of the REST API endpoints, accuracy of the AI parsing and scoring pipeline, edge case handling for invalid inputs, and verification of the explainability output. Tests were designed to reflect realistic recruiter workflows and representative resume and job description content from the software engineering domain.

## 6.1Test Cases

| Test ID | Test Description | Input | Expected Output | Actual Output | Status |
|---|---|---|---|---|---|
| TC-01 | Upload a valid PDF resume | PDF file ≤ 10 MB with clear sections | HTTP 200; parsed name, email, skills, experience returned | Correct fields extracted and returned in ResumeResponse | PASS |
| TC-02 | Upload a valid DOCX resume | DOCX resume with skills and education sections | HTTP 200; structured data extracted | All fields correctly parsed from DOCX | PASS |
| TC-03 | Upload a valid TXT resume | Plain-text resume | HTTP 200; structured data extracted | Fields extracted via regex matching | PASS |
| TC-04 | Upload file with invalid extension (.png) | PNG image file | HTTP 400; error Unsupported file type | Correct HTTP 400 with error message | PASS |
| TC-05 | Upload file exceeding 10 MB size limit | PDF file > 10 MB | HTTP 400; error File too large | Correct HTTP 400 with size error | PASS |
| TC-06 | Submit a job description with all skill types | JD with required and preferred skills, experience, education | HTTP 200; parsed required_skills list returned | All skills extracted including multi-word terms | PASS |

| | | | | | |
|---|---|---|---|---|---|
| TC-07 | Match a strong candidate to a well-aligned job | Resume with 8/10 required skills, 4 years exp, Bachelor's | match_score ≥ 60%; prediction Selected | Score: 78.4%, prediction: Selected | PASS |
| TC-08 | Match a weak candidate to a demanding job | Resume with 1/10 required skills, 0 years exp | match_score < 60%; prediction Rejected | Score: 22.0%, prediction: Rejected | PASS |
| TC-09 | Verify weighted formula correctness | Known feature scores | match_score = 100% | Computed: 100% | PASS |
| TC-10 | Verify certification bonus cap | All primary scores = 1.0, cert_score = 1.0 | Final score capped at 100% | Score: 100% | PASS |
| TC-11 | Explainability report structure | Any valid resume-job match | Explanation contains required fields | All keys present in response JSON | PASS |
| TC-12 | Batch matching with multiple resumes | 5 resumes uploaded | 5 ScreeningResults returned | Correct batch results returned | PASS |
| TC-13 | Candidate ranking by score | 4 resumes matched | Rankings descending order | Correctly sorted | PASS |
| TC-14 | Candidate comparison | resume_id_a, resume_id_b | CompareResponse generated | Full comparison generated | PASS |
| TC-15 | Comparison diff >5% | A:80%, B:55% | Recommend A | Correct recommendation | PASS |
| TC-16 | Comparison diff <5% | A:72%, B:70% | Closely matched | Correct recommendation | PASS |
| TC-17 | Delete resume | DELETE API | HTTP 200 | Deleted successfully | PASS |
| TC-18 | Retrieve non-existent resume | Invalid IDs | HTTP 404 | Correct response | PASS |
| TC-19 | Resume no skills | No known skills | Empty skills list | Handled gracefully | PASS |

| | | | | | |
|---|---|---|---|---|---|
| TC-20 | Health check endpoint | GET /api/health | HTTP 200 | Healthy response | PASS |

**Table 6.1 Sample Test Cases**

## 6.2Results and Discussions

The system testing phase yielded successful outcomes across all 20 test cases, confirming that HireLens AI's core functional requirements have been correctly implemented and behave as expected under both normal and edge-case conditions. The results demonstrate that the system correctly handles file validation, multi-format parsing, feature scoring, weighted aggregation, explainability generation, batching, ranking, and candidate comparison workflows.

**Key Results**

- All 20 test cases passed, achieving a 100% functional test pass rate.
- Resume parsing successfully extracted structured fields from all three supported formats (PDF, DOCX, TXT).
- The weighted scoring formula was verified to produce mathematically correct results, including the certification bonus and the 1.0 cap on the final score.
- The selection threshold of 60% correctly separated strongly aligned candidates from poorly aligned ones in test cases TC-07 and TC-08.
- Explainability reports were correctly generated for all matching scenarios, containing all required fields including feature contributions, natural language summary, matched skills, and missing skills.
- Batch matching correctly processed multiple resumes in sequence and returned ordered results.
- Candidate ranking was verified to sort results in strict descending order by match score.
- Candidate comparison correctly generated differentiated recommendations based on score difference thresholds.
- Edge cases (invalid file type, oversized file, non-existent IDs, resumes with no skills) were handled gracefully with appropriate HTTP error codes.

**Discussion**

The system performs reliably for the use cases it was designed to address. The primary limitation observed during testing is that the skill extraction algorithm is bounded by the curated KNOWN_SKILLS ontology — novel or domain-specific skills not present in this list will not be detected. This is a deliberate design trade-off that prioritises explainability and determinism over coverage; a keyword-based extraction approach produces consistent, auditable results, while a model-based approach (e.g., using spaCy NER) would be more flexible but harder to debug. A second observed limitation is that experience year extraction from resumes that describe experience implicitly (e.g., through a list of dated roles without explicit "X years of experience" phrasing) relies on counting work date ranges, which may produce rough estimates rather than precise values.

## 6.3 Performance Evaluation

**Processing Speed:**

The end-to-end processing time for a single resume-job matching operation — from the moment the API receives the POST /api/match_resume request to the moment the ScreeningResultResponse is returned — was consistently measured at under 500 milliseconds for resumes of typical length (1–3 pages) when running on a development machine with a 2.5 GHz quad-core processor and 16 GB RAM. This is well within the 5-second non-functional requirement (NFR-01). The primary time cost is PDF text extraction via pdfplumber for complex, multi-column PDF layouts, which may occasionally reach 1–2 seconds for highly formatted documents.

**Text Extraction Accuracy:**

The regex-based information extraction pipeline produced high-accuracy results for standard resume structures. Email extraction (regex-based) achieved near-perfect accuracy. Phone extraction correctly identified standard international and domestic formats. Skill extraction correctly identified all skills present in the KNOWN_SKILLS ontology across all test documents, with no false positives observed on test resumes. Experience year extraction correctly parsed explicit phrasing (e.g., "5+ years of experience") in all test cases.Performance Efficiency and Response Time

**Scoring Accuracy:**

The weighted linear combination scoring model produces monotonically correct results: candidates with objectively better alignment across all four dimensions consistently receive higher scores. The configurable threshold and weights allow the system's precision/recall trade-off to be tuned for different hiring contexts; a more selective threshold (e.g., 0.70) would reduce the "Selected" pool to only top-tier candidates, while a lower threshold (e.g., 0.50) would increase recall at the cost of precision.

**Scalability Considerations:**

In the current implementation, resume parsing and matching are performed synchronously within FastAPI request handlers. For production use with hundreds of concurrent requests, these operations should be offloaded to a background task queue (e.g., Celery with Redis) to prevent blocking the ASGI event loop. The modular AI pipeline architecture (each module is an independent Python package) facilitates this migration without requiring changes to the matching logic itself.

**Explainability Quality:**

The natural language summaries generated by the explainability engine were evaluated qualitatively against the underlying feature scores and found to accurately reflect the quantitative outcomes in all 20 test cases. The feature contribution breakdown correctly identifies the dominant and weakest factors in each screening result, providing actionable insight to the recruiter.

.

# CHAPTER 7

# CONCLUSION & FUTURE ENHANCEMENTS

HireLens AI successfully demonstrates that it is possible to build a fully automated resume screening system that is simultaneously powerful, interpretable, and practically usable without requiring domain expertise in artificial intelligence from its users. The project achieves its primary objective: replacing the black-box opacity of conventional applicant tracking systems with a transparent, weighted multi-dimensional evaluation framework that exposes its entire decision-making logic to the recruiter through feature scores, natural language summaries, matched and missing skill lists, and structured strengths and weaknesses. The system's six-stage AI pipeline — parsing, job analysis, feature extraction, score aggregation, prediction, and explainability — is implemented as a modular, independently testable collection of Python packages, making the system maintainable and extensible. The React frontend delivers an intuitive, multi-step user experience that requires no training to operate, and the FastAPI backend provides a clean, type-safe REST API that serves as a reliable integration boundary between the AI logic and the presentation layer.

Through the course of this project, several important insights were gained. First, explainability is not an add-on feature but a foundational architectural principle that must be designed into the system from the outset — the decision to compute independent, named feature scores rather than a single aggregate embedding made the explainability layer straightforward to implement. Second, the configurability of the matching weights and selection threshold is a critical practical requirement, as different organisations and roles require different trade-offs between skills, experience, and education. Third, the modular pipeline architecture proved valuable during development, allowing individual components (e.g., the education scoring logic) to be refined and tested without affecting the rest of the system.

## Future Enhancements

- **Semantic Similarity Integration:** The current implementation relies on exact keyword matching for skill comparison. Integrating the already-installed sentence-transformers library would enable cosine similarity between sentence embeddings, allowing the system to identify semantically equivalent skills (e.g., "ML" and "machine learning", or "TypeScript" and "JavaScript") that differ syntactically but are functionally equivalent, significantly improving recall.

- **SHAP and LIME Integration:** Although shap and lime are included as dependencies, the current explainability implementation uses a custom rule-based contribution computation. Integrating SHAP values or LIME explanations would provide mathematically rigorous, game-theoretically grounded feature attributions, making the system suitable for deployment in high-stakes environments with auditability requirements.
- **Named Entity Recognition with spaCy:** The installed spaCy library could replace the current heuristic-based name, experience duration, and education extraction with trained NER models, improving extraction accuracy on unconventionally formatted resumes.
- **Asynchronous Batch Processing:** For production scale, the batch matching endpoint should be implemented using background tasks (FastAPI BackgroundTasks or Celery) to process large resume pools without blocking the API server or imposing long HTTP response times on the client.
- **User Authentication and Multi-Tenancy:** Adding user authentication (e.g., via OAuth 2.0 or JWT tokens) would allow multiple recruiters within an organisation to maintain separate job and resume pools, with role-based access controls for administrative functions.
- **Production Database Migration:** Replacing SQLite with PostgreSQL would enable concurrent write access, better performance at scale, and compatibility with managed cloud database services for production hosting.
- **Bias Detection and Fairness Monitoring:** A future enhancement of significant ethical importance would be the integration of fairness metrics to monitor screening decisions for demographic bias. By tracking selection rates across protected attributes (where such data is appropriately collected and consented), HireLens AI could provide bias alerts and help organisations ensure equitable hiring practices.
- **Resume Scoring Calibration Interface:** Providing a recruiter-facing calibration interface to adjust matching weights per job role (e.g., increasing the experience weight for senior roles and the skills weight for junior technical roles) would make the system significantly more flexible for diverse hiring contexts.
- **Applicant-Facing Feedback Portal:** A candidate-facing interface where applicants can receive anonymised feedback (matched skills, missing skills, suggested improvements) would transform HireLens AI from a pure recruiter tool into a two-sided career development platform.
- **CV Improvement Suggestions:** Building on the existing missing skills analysis, the system could generate structured learning recommendations (e.g., "Acquiring Docker and Kubernetes would improve your match score for this role from 45% to 72%"), adding significant value for candidates and organisations running internal mobility programmes.

# CHAPTER 8

# REFERENCES


1. Yu, K., Guan, G., & Zhou, M. (2005). *Resume Information Extraction with Cascaded Hybrid Model*. Proceedings of the 43rd Annual Meeting of the Association for Computational Linguistics (ACL 2005), pp. 499–506.

2. Bogen, M., & Rieke, A. (2018). *Help Wanted: An Examination of Hiring Algorithms, Equity, and Bias*. Upturn. Retrieved from https://upturn.org/work/help-wanted/

3. Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). *"Why Should I Trust You?": Explaining the Predictions of Any Classifier*. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 1135–1144.

4. Lundberg, S. M., & Lee, S.-I. (2017). *A Unified Approach to Interpreting Model Predictions*. Advances in Neural Information Processing Systems (NeurIPS 2017), 30, pp. 4765–4774.

5. Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. Proceedings of NAACL-HLT 2019, pp. 4171–4186.

6. Reimers, N., & Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP 2019), pp. 3982–3992.

7. Faliagka, E., Tsakalidis, A., & Tzimas, G. (2012). *An Integrated e-Recruitment System for Automated Personality Mining and Applicant Ranking*. Internet Research, 22(5), pp. 551–568.

8. Triantaphyllou, E. (2000). *Multi-Criteria Decision Making Methods: A Comparative Study*. Springer, Dordrecht. ISBN: 978-1-4757-3157-6.

9. Rajaraman, A., & Ullman, J. D. (2011). *Mining of Massive Datasets* (Chapter 9: Recommendation Systems). Cambridge University Press.

10. Peng, R. D., & Matsui, E. (2015). *The Art of Data Science*. Leanpub.

11. FastAPI Documentation. Sebastián Ramírez. (2024). Retrieved

from https://fastapi.tiangolo.com/

**12.** SQLAlchemy Documentation. (2024). Retrieved from https://docs.sqlalchemy.org/en/20/

**13.** React Documentation. Meta Open Source. (2024). Retrieved from https://react.dev/

**14.** Vite Documentation. (2024). Evan You. Retrieved from https://vitejs.dev/

**15.** pdfplumber Documentation. Jeremy Singer-Vine. (2024). Retrieved from https://github.com/jsvine/pdfplumber

**16.** spaCy Documentation. Explosion AI. (2024). Retrieved from https://spacy.io/

**17.** sentence-transformers Documentation. (2024). Retrieved from https://www.sbert.net/

**18.** SHAP Documentation. Scott Lundberg. (2024). Retrieved from https://shap.readthedocs.io/

**19.** European Commission. (2016). *Regulation (EU) 2016/679 (GDPR) — Article 22: Automated Individual Decision-Making, Including Profiling*. Official Journal of the European Union.

**20.** Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. ISBN: 978-0-262-03561-3.